최종 보고서

Oct 24, 2026 · @CV실습04

프로젝트 개요

항목	내용
프로젝트	오픈스페이스 알박기 감지 시스템 (한양대 ERICA 융합교육관 스터디룸 14개)
기간	2026년 10월 13일 ~ 10월 23일 기획 10/13 · 1차 회의 10/15 · 2차 회의 10/21 · 최종 발표 10/22 · 발표 후 보강 10/23
팀	알박기 척결
팀원과 역할	황예성: 총괄·기획, 발표 자료 제작과 발표, 프로젝트 전반 검수 이창민: 라즈베리파이 카메라 송출과 실측 박서준: 서버 판정 프로그램, YOLO 검증·실험
최종 결과물	발표 자료, 판정 모델(YOLO26s v2), 통합 실행 프로그램, MVP 관제 대시보드, GitHub 10_Project

문제해결과정

프로젝트는 "30분 자리 비움 규칙은 있지만, 지켜졌는지 확인할 수 없다"는 문제에서 출발했다. 1차 회의(10/15)에서는 YOLO11s로 사람과 테이블을 찾고, 테이블 박스를 가로·세로 30% 넓힌 감지 영역 안에 사람이 있는지로 재석을 판단하기로 했다. 엎드린 사람은 못 잡을 것이라는 우려가 나와 회의 중에 사진 6장으로 바로 확인했고, 모델 6종이 모두 6장을 잡아 사전학습 모델로 충분하다고 봤다. 이후 실제로 만들고 시험하면서 아래 문제를 차례로 만났다.

검증 방법. 룸 14개를 실제로 다 찍을 수 없어서, 스터디룸 14개 디지털 트윈에 가상 학생을 넣고 룸별 천장 카메라 화면을 YOLO 서버로 판정했다. 앉은 사람은 98.2%, 룸 판정은 92.0%(112장)로 좋아 보였지만, 가장 중요한 엎드린 사람이 48.9%로 실제 사진(6/6)과 정반대였다. GPU 없이 렌더링하면 4명이 있는 룸을 0명으로 판정하기도 했다. 점수가 모델 성능이 아니라 그래픽의 한계를 재고 있었고, 가상 화면을 실제에 맞추는 시간이 실제 룸을 한 번 찍는 것보다 컸다. 그래서 하루 만에 시뮬레이션을 멈추고, 실제 룸 천장에 카메라를 달아 22분을 찍어 검증했다.

테이블 검출. 실제 영상에서는 1차 회의의 전제가 무너졌다. 사전학습 모델은 책상을 식탁(dining table)으로만 배워, 천장에서 내려다본 우리 테이블을 거의 찾지 못했다(YOLO26s 2%). 테이블을 못 찾으면 감지 영역을 만들 수 없다. 모델을 키우면 해결될지 보려고 사전학습 모델 16종을 같은 조건으로 비교했지만, 테이블과 엎드린 사람을 둘 다 잡는 모델은 없었다. 테이블을 잡는 YOLO11l은 엎드린 사람이 31%, 엎드린 사람을 잡는 YOLOv10s는 테이블이 8%였다. 모델 크기가 아니라 데이터의 문제라고 보고 파인튜닝으로 방향을 바꿨다.

파인튜닝. 우리 룸에서 만든 데이터 727장(학습 637 + 검증 90)으로 학습했는데, 1차 학습은 테이블 위주 데이터라 테이블은 100%가 됐지만 사람을 잊어버렸다(사람 100%→68%, 가방 89%→14%). 사람 데이터를 같이 넣고 학습률을 낮춘 v2에서 둘 다 해결했다(테이블 2%→100%, 엎드린 사람

92.9%→100%). 공정하게 평가하려고 천장 영상의 뒤 20%와 웹캠 영상은 학습에 넣지 않고 시험에만 썼다.

모델	파라미터	테이블	엎드린 사람	GPU 1장
YOLO26n (전)	2.6M	3%	65.5%	17.9ms
YOLO26s (전)	10.0M	2%	92.9%	21.0ms
YOLO11s v2	9.5M	100%	98.8%	17.7ms
YOLO26s v2 (최종)	10.0M	100%	100%	18.1ms
YOLO26m v2	21.9M	100%	100%	21.6ms

학습에 안 쓴 영상 기준, GPU는 팀 노트북 RTX 5060·입력 960·30회 중앙값.

엎드린 사람 데이터. 엎드린 사람 사진이 부족해, 생성형 데이터로 채울 수 있는지 시험했다. 직접 찍은 빈 룸 사진의 의자 자리 4곳에 Stable Diffusion 인페인팅(사진 일부만 새로 그리는 기법)으로 엎드린 사람을 그려 넣고, CLIP으로 먼저 거른 뒤 눈으로 확인했다. 지시문과 옷차림을 바꿔 세 번에 걸쳐 80장을 만들었고, 그중 엎드린 장면은 40장이다. 결과적으로 학습에는 쓰지 않았다. YOLO26s가 엎드린 장면 40장을 모두 잡아 새로 배울 것이 없었고, 진짜 약점이던 천장 각도의 테이블은 합성 사진으로 풀 수 없었으며, 1장 만드는 데 약 70초가 걸렸다. 대신 약점을 찾는 데는 쓸모가 있었다. 후드를 쓰고 엎드린 사람은 더 큰 YOLO26l도 신뢰도 0.14로 놓쳤다. 그래서 시범 수집 때 후드 쓰고 엎드린 장면을 먼저 모으기로 했다.

감지 영역. 테이블을 잡게 된 뒤에는 사각형 박스가 통로까지 덮어, 지나가는 사람도 재석으로 세는 문제가 남았다. 그래서 SAM으로 테이블 테두리만 따고, 짧은 변의 15%만큼 부풀려 감지 영역으로 썼다. 테이블 끝자리 사람까지 잡으려고 영역을 더 넓히자 유리벽 너머 옆방 사람까지 잡혔다. 그래서 영역은 +15%로 두고 사람 박스의 겹침 기준을 낮췄다. 25%와 15%에서는 마지막 사람이 나가는 순간 사람이 있는데도 룸을 비었다고 본 순간이 있었고, 10%에서는 없어서 최종 기준을 10%로 정했다.

판정 위치와 카메라. 1차 회의에서는 룸마다 라즈베리파이를 두고 그 안에서 직접 추론하는 안을 검토했다. 라즈베리파이 4B에서 같은 YOLO26s v2를 영상 659장으로 실측해 보니 정확도는 같았지만, 1장에 2,832ms(초당 0.35장)가 걸려 1초마다 판정할 수 없었다. GPU 서버(18.1ms)보다 약 156배 느리다. 또 최고 81.3°C까지 올라 하루 종일 돌리기 어렵고, 14대를 따로 관리해야 하며 초기 비용도 약 240만 원으로 PC 한 대보다 컸다. 클라우드 GPU는 빠르지만 월 약 60만 원이 들고 영상이 학교 밖으로 나간다. 그래서 GPU가 있는 교내 PC 1대에서 14개 룸을 모두 판정하기로 했다.
카메라도 기존 CCTV는 한 화면에 여러 룸이 겹치고 스터디룸 안에는 없었으며, 앱 전용 홈캠은 RTSP로 영상을 받을 수 없고 영상이 제조사 클라우드를 거쳤다. 그래서 RTSP로 교내망에서 직접 받을 수 있는 Tapo C210을 골랐다.

타깃과 비용. 교수님 피드백으로 "누구에게 파는가"와 "비용이 정말 최소인가"를 다시 정리했다. 고객은 학교 운영 담당, 사용자는 학생인 B2B로 정하고, 모든 선택을 ① 정확도·속도 ② 비용 ③ 운영·유지보수 ④ 개인정보로 판단했다. 처음 비용에는 PoE 스위치, 업체 시공, 현황판 모니터가 들어 있었는데, 와이파이 카메라로 바꿔 PoE와 시공을 빼고 현황판은 고도화 단계로 미뤘다. "오픈소스도 공짜가 아니다"라는 지적에 따라 점검·고장 수리·재학습 인건비를 룸당 월 1.5

만 원 구독에 넣었고, 초기 207만 원, 연 252만 원(학생지원비의 0.02%)으로 정리했다. 서버 사양은 노트북 실측(1장 18.1ms)을 GPU 연산 성능 비율로 환산해 최소 GTX 1060급으로 정했다.

빈 책상. 판정 규칙에도 빈틈이 있었다. 사람이 없는 시간만 재다 보니, 짐까지 챙겨 나간 빈 책상도 노랑으로 30분을 기다렸다. 노랑은 짐 주인이 돌아올 수 있으니 기다린다는 뜻인데, 짐이 없으면 기다릴 이유가 없다. 그래서 감지 영역 안의 짐(가방·노트북·책 등 11종)도 함께 보고, 사람도 짐도 10초 연속 없으면 바로 초록으로 바꾸게 했다. 파인튜닝 모델은 가방을 거의 못 잡아서, 가방을 놓쳐 짐 있는 책상이 빈 책상으로 보이지 않도록 가방류의 신뢰도 기준은 0.10으로 낮췄다.

설명하기 어려운 결과. 판정 방식과 도입 효과는 말로 설명하기 어려웠다. 그래서 실제 룸 영상 22분을 더미 데이터로 삼아 14개 룸을 가상으로 운영하는 관리자 대시보드(MVP)를 만들었다. 저녁 식사 시간 룸 활성화율이 도입 전 22%에서 도입 후 50%로 바뀌는 것을 한 화면에서 보여 줄 수 있게 됐다.

발표 후 보강. 최종 발표(10/22) 당일 받은 피드백을 반영해 프로젝트를 보강했다. 발표 시점에는 판정 코드, 모델, 대시보드가 따로 있어, 카메라 영상을 받아 판정하고 신호등까지 보내는 프로그램이 없었고 룸 14개 동시 처리도 시험하지 못한 상태였다. 그래서 카메라 여러 대 → 판정 → 신호등·기록·현황 화면을 한 번에 돌리는 통합 실행 프로그램을 만들고, 영상 하나를 룸 14개처럼 복제해 1초마다 판정했다. 한 바퀴(14개 룸) 처리 시간은 중앙값 453ms로, 1초 간격 판정이 가능했다. 현황 화면이 카메라 사진을 2초마다 한 장씩 불러와 끊겨 보이는 문제는, 판정은 1초마다 그대로 하고 화면만 실시간 영상으로 보내 해결했다. 박스가 최대 1초 늦게 따라오므로, 모자이크는 사람 박스보다 25% 넓게 씌웠다.

카메라 각도와 성능 측정. 보강 중 웹캠으로 성능을 확인하다가 새 문제를 찾았다. 파인튜닝 모델이 정면 웹캠 앞 사람을 14장 중 2장만 찾았다(기본 YOLO26s는 14장 모두). 천장에서 내려다본 사진으로만 학습해, 학습한 각도에는 강해진 대신 다른 각도에는 약해진 것이다. 실제 설치는 천장 카메라라 운영에는 파인튜닝 모델을 그대로 쓰지만, 카메라 각도가 바뀌면 그 각도의 데이터로 다시 학습해야 한다는 뜻이기도 하다. 또 웹캠 확인 화면의 처리량(1초에 룸 14~17개)이 발표의 초당 55장보다 낮게 나왔는데, 판정과 화면 그리기를 한 프로그램에서 같이 하고 노트북이 배터리 전원이었기 때문이었다. 화면 없이 판정만 하면 1장 29.4ms(1초에 약 34개)로, 가장 불리한 조건에서도 룸 14개를 1초 안에 판정할 수 있었다.

남은 문제. 현장 데이터가 룸 1곳(영상 22분)뿐이고, 실제 카메라 14대(RTSP)와 GTX 1060급 장비로 한 시험은 아직 하지 않았다. 그래서 학교 협조를 받아 7일 동안 운영과 같은 천장 각도로 현장 데이터를 모으되 후드 쓰고 엎드린 장면처럼 약한 장면을 먼저 모으고, 재학습·시험에 합격하면 적용하기로 했다.

진행 사항

프로젝트 진행 흐름

프로젝트는 기획(10/13)부터 발표 후 보강(10/23)까지 아래 순서로 진행했다. 팀원마다 맡은 실험을 동시에 진행하고, 회의에서 결과를 모아 다음 방향을 정했다. 각자 한 일의 자세한 내용은 뒤의 개인 연구 보고에 있다.

날짜	단계	한 일	담당
10/13	기획	주제를 오픈스페이스 알박기로 좁히고 스터디룸 14개를 현장 조사했다. 핵심 문제를 "30분 규칙은 있는데 지켜졌는지 확인할 수 없다"로 정하고 6 Pager 기획서(문제분석지·PR·FAQ 19문항)를 썼다.	황예성
10/15	1차 회의	역할을 나누고 판정 방식(YOLO11s, 테이블 박스 +30% 감지 영역, 10초 가림 보정, 3색 규칙)과 판정 코드(MVP)를 정했다. "엎드린 사람은 못 잡는다"는 우려는 회의 중 사진 6장으로 확인했다(6종 모두 6/6).	전원 (판정 코드· 검증: 황예성)
10/15~17	카메라 송출	라즈베리파이 카메라 영상 송출(HTTP/MJPEG), PC에서 받은 영상으로 YOLO 추론 테스트, 수신 스트림 녹화 기능을 만들었다.	이창민
10/15~17	3D 검증과 중단	스터디룸 14개 3D 디지털 트윈으로 시험했지만 엎드린 사람이 48.9%(실제 6/6)로 나와 중단하고, 실제 룸 천장 카메라 영상(22분) 검증으로 바꿨다.	황예성
10/18	모델·데이터	사전학습 YOLO 16종을 비교했지만 테이블과 엎드린 사람을 둘 다 잡는 모델이 없었다. 학습 데이터 727장을 만들고(테이블 라벨 181장 검수), 1차 학습 실패 뒤 v2로 파인튜닝했다(테이블 2%→100%).	황예성
10/18~21	판정 프로그램 ·검증	판정 프로그램(server.py)과 다각형 감지 영역을 만들고, 사전학습 15종 비교, 정답 659장 프레임 검증(v2 학습 외 구간 131/132), 생성형 데이터 80장 실험, 공개 데이터셋 10종 조사, 실시간 수신 화면을 진행했다.	박서준
10/20~21	비용·사양· 감지 범위	B2B 비용(초기 207만 원, 구독 연 252만 원 = 학생지원비의 0.02%), 서버 최소 사양(GTX 1060 급), SAM 감지 범위(+15%, 겹침 기준), 시스템 컨텍스트 다이어그램, MVP 관제 대시보드를 만들었다.	황예성
10/20~21	라즈베리파이 실측	송출을 H.264 RTSP로 바꾸고, 라즈베리파이 4B 에서 YOLO26s v2를 직접 돌리는 벤치마크를 했다 (1장 2,831.8ms, 최고 81.3°C).	이창민
10/21	2차 회의	실험 결과를 모아 판정은 교내 PC 1대, 모델은 YOLO26s v2, 감지 범위는 SAM +15%로 정하고 라즈베리파이와 클라우드 GPU는 제외했다. 발표 자료를 팀원 피드백으로 고쳤다.	전원

날짜	단계	한 일	담당
10/22	최종 발표	리허설 녹음으로 발표문을 다듬고 빈 책상 바로 초록 규칙을 넣었다. 발표 자료 수정본과 예상 질문을 정리하고 최종 발표를 했다.	황예성(발표) 박서준 (수정본·예상 질문)
10/23	발표 후 보강	발표 당일 받은 피드백을 반영해, 카메라 영상 → 판정 → 신호등·현황 화면을 한 번에 돌리는 통합 실행 프로그램(룸 14개 한 바퀴 453ms)과 웹캠 성능 확인 창을 만들고 저장소와 보고서를 정리했다.	황예성

주요 논의와 결론

주제	내용	결론
추론 위치	라즈베리파이 내부 추론 vs 서버 판정	라즈베리파이는 1초 간격 불가· 과열로 제외, 교내 PC 1대
모델 크기	nano는 엎드린 사람 65.5%, m·l은 정확도 같고 느림	YOLO26s (GPU에서는 n과 속도 차이 거의 없음)
학습 여부	1차에서는 사전학습으로 충분하다고 봤으나 테이블을 못 잡음	파인튜닝하기로 변경
감지 영역	사각형 박스는 통로까지 덮고, 더 넓히면 유리벽 너머 옆방이 잡힘	SAM 테두리 +15%, 겹침 10%
타깃	교수님 피드백 "타깃을 정하라"	B2B: 고객은 학교, 사용자는 학생. 판단 기준 4가지(정확도·속도, 비용, 운영, 개인정보)
비용	교수님 피드백 "비용이 최소인지, 오픈소스도 공짜가 아니다"	초기 207만 원, 구독 연 252만 원 (학생지원비의 0.02%)
합성 데이터	기존 모델이 이미 잡는 장면이라 학습에 새로 줄 정보가 없음	학습 제외, 약점 찾기에 활용

시도했다가 제외한 것

직접 해 보고 숫자로 판단해 버린 안이다. 걸린 기준은 학교 입장의 판단 기준 번호다(① 정확도·속도 ② 비용 ③ 운영·유지보수 ④ 개인정보).

시도	결과	걸린 기준	대신 고른 것
기존 CCTV 활용	한 화면에 여러 룸이 겹치고, 스터디룸 안에는 CCTV가 없음	①	룸마다 카메라 1대
가정용 홈캠(앱	RTSP로 영상을 받을 수 없고 영상이 제조사	④	RTSP 지원 Tapo

시도	결과	걸린 기준	대신 고른 것
전용)	클라우드를 거침		C210
3D 디지털 트윈 검증	엎드린 사람 48.9%(실제 사진은 6/6). GPU 없이 렌더링하면 4명을 0명으로 판정	①	실제 사진·실제 룸 영상으로 검증
룸별 라즈베리파이 추론	1장 2,832ms(초당 0.35장), 최고 81.3°C, 14대 초기 약 240만 원	①②③	교내 PC 1대(GPU 1장 18.1ms)
클라우드 GPU	월 약 60만 원, 영상이 학교 밖으로 나감	②④	교내 PC 1대
사전학습 모델 그대로 사용	16종 중 테이블과 엎드린 사람을 둘 다 잡는 모델 없음(기존 YOLO11s 엎드림 6%)	①	우리 영상으로 파인튜닝
1차 파인튜닝 (테이블 위주)	테이블은 100%지만 사람을 잊음(사람 100%→68%, 가방 89%→14%)	①	사람 데이터를 넣고 학습률을 낮춘 v2
Stable Diffusion 합성 데이터로 학습	합성 80장(엎드린 장면 40장)을 기존 모델이 이미 잡아 새로 줄 정보가 없었고, 진짜 약점(천장 각도의 테이블)은 합성 사진에 없음	①②	현장 데이터. 합성 사진은 약점 찾기에 활용
더 큰 모델 YOLO26m·l	정확도는 같고 CPU에서 2.7~3.4배 느림	①	YOLO26s
더 작은 모델 YOLO26n	엎드린 사람 65.5%, GPU에서는 s와 속도 차이 없음	①	YOLO26s
YOLO 테이블 박스를 30% 넓힌 감지 영역	사각형이라 비스듬한 테이블 옆 통로까지 덮어 지나가는 사람도 재석으로 셈	①	SAM 테이블 테두리 +15%
감지 영역을 더 넓히기	유리벽 너머 옆방 사람까지 잡힘	①	겹침 기준 25% → 10%
PoE 스위치·업체 시공·현황판 모니터	와이파이 홈캠이라 불필요, 현황판은 고도화 단계	②	교내 와이파이 연결, 전기 공사만 (총 30만 원)

팀 결정 사항 전체

분야	결정	이유
방향	B2B: 고객은 학교 운영 담당, 사용자는 학생	교수님 피드백 "타깃을 정하라". 학생은 앱 설치·입력 없이 색만 봄

분야	결정	이유
판단 기준	① 정확도·속도 ② 비용 ③ 운영·유지보수 ④ 개인정보	모든 선택을 학교 입장에서 비교하기 위해
카메라	룸마다 Tapo C210 1대(대당 45,900원), RTSP로 교내망 직접 수신, 교내 와이파이 연결, 천장에 고정해 책상이 잘 보이도록 설치	룸 하나 = 화면 하나, 업체 시공 불필요
신호등	가로형 3색 신호등을 문이나 문 위에 부착. 빨강 사용중, 노랑 30분 미만, 초록 30분 이상 또는 빈 책상(사람도 짐도 없음)	복도에서 바로 보이게. 초록이면 짐을 룸 앞 바구니로 옮기고 사용
판정 서버	교내 PC 1대: 최소 GTX 1060급, 권장 GTX 1070급 + 6코어 라이젠 5 5600급 + 램 16GB(약 50만 원)	룸 14개 × 1초 1장 = 1장 71ms 안. CPU는 카메라 14대 영상 디코딩 과부하 방지
모델	YOLO26s v2(파인튜닝), 입력 960, 엎드린 사람 기준값 0.35	테이블·사람·엎드린 사람 모두 100%, NMS-free로 후처리 없음
데이터	727장(학습 637 + 검증 90). 천장 영상 앞 80%는 학습, 뒤 20%와 웹캠 영상은 시험에만 사용	학습에 안 쓴 영상으로 공정하게 평가
평가 지표	사람·엎드린 사람 검출률, 테이블 IoU 0.5 이상 비율, 1장당 처리 시간, 파라미터 수	사람을 놓치면 초록불이 켜져 짐을 치우게 됨. 속도·크기는 서버 비용과 직결
감지 영역	SAM 테이블 테두리를 짧은 변의 15%만큼 부풀림, 사람 박스가 10% 이상 겹치면 재석. 설치 때와 테이블이 움직였을 때만 다시 딴	통로·옆방 오탐 방지
판정 보정	사람이 10초 넘게 안 보여야 부재, 2초 연속 보여야 재석	가려짐으로 인한 오판과 순간 오검출 방지
개인정보	얼굴 식별 없음, 영상 저장 없음, 교내망 밖 전송 없음, 색이 바뀐 순간 스냅샷만 모자이크해 1일 보관, 기록은 시각·룸·색만, 공개 자료도 모자이크	학생을 찍는 시스템이라 처음부터 설계에 포함
비용	초기 207만 원(카메라 64만, PC 50만, 신호등 49만, 전기 공사 30만, 부자재 14만). 구독 룸당 월 1.5만 원(점검·고장 수리·재학습 인건비 포함, 출장비 제외). 전기세는 학교 부담	오픈소스도 공짜가 아님. 룸 단위라 일부 룸부터 도입 가능. 연 252만 원 = 학생지원비의 0.02%
시범 운영	학교 협조 → 7일 현장 수집(운영과 같은 천장 각도, 후드 쓰고 엎드린 장면 먼저) → 재학습·	현장 데이터가 룸 1곳뿐이라 실증으로 채움

분야	결정	이유
	테스트 → 합격 시 적용	
불합격·중단 기준	불합격: 관찰 300회 중 잘못된 초록 1건 이상 또는 빈 룸 놓침 2% 초과. 중단: 개인정보 검토 불통과 또는 분실·다툼 문의 주 3건 이상	잘못 켜진 초록 한 번이 곧 민원
고도화	1층 현황판, 짐 검출	MVP 범위 밖, 초기 비용 최소화

추후 계획

시범 운영 계획

최종 발표 이후에는 실제 룸에서 시범 운영을 거쳐 적용하는 것을 목표로 한다.

- 학교 협조: 시설팀에 카메라 설치 가능 여부를 묻고, 운영 승인과 개인정보 검토를 받는다.
- 7일 현장 수집: 운영과 같은 천장 각도로 14개 룸을 찍되, 후드 쓰고 엎드린 장면처럼 약한 장면을 먼저 모은다.
- 재학습과 시험: 현장 데이터로 다시 학습하고, 14개 룸을 하루 3번씩 7일(약 300회) 직접 관찰해 판정과 비교한다. 합격하면 14개 룸에 적용한다.
- 장비 실측: 실제 카메라 14대(RTSP)와 GTX 1060급 PC로 통합 실행 프로그램을 돌려, 1초 간격 판정을 다시 확인한다.

기타

- 시범 운영 불합격 기준을 미리 정했다: 14개 룸을 하루 3번씩 7일, 약 300회 직접 관찰해 판정과 비교하고, 주인이 있는데 초록이 켜진 경우 1건 이상 또는 빈 룸을 놓친 경우 2%(6건) 초과면 재학습한다.
- 중단 기준: 개인정보 검토 불통과 또는 짐 분실·다툼 문의 주 3건 이상이면 장비를 철거하고 기록을 모두 삭제한다.
- 남은 문제: 현장 데이터가 룸 1곳(영상 22분)뿐이고, 실제 카메라 14대(RTSP)와 GTX 1060급 실측은 아직 못 했다. 30분 직전에 잠깐 들르면 타이머가 초기화된다.
- 고도화: 1층 현황판(층을 돌지 않고 빈 룸 확인), 짐 검출

황예성 개인 연구 보고: 판정 모델·감지 범위 개선과 시스템 통합

담당: 프로젝트 총괄·기획, 발표 자료 제작과 발표, 프로젝트 전반 검수. 실험으로는 3D 디지털 트윈 검증, 사전학습 YOLO 16종 비교와 파인튜닝(학습 데이터 727장), SAM 감지 범위, 비용·서버 사양 계산, MVP 관제 대시보드, 통합 실행 프로그램을 맡았다. 코드와 결과는 개인 저장소 MODU_CV(https://github.com/hwang-ye-song/MODU_CV/tree/main/프로젝트_오픈스페이스_알박기감지)와 팀 저장소 10_Project에 있고, 이 보고서의 수치는 그 기록에서 가져왔다.

요약

1주차에는 융합교육관 스터디룸 14개를 직접 조사해 "30분 규칙은 있는데, 지켜졌는지 확인할 수 없다"는 문제를 정의하고, 6 Pager 기획서와 판정 방식(테이블 박스 +30% 감지 영역, 10초 가림 보정, 3색 규칙)을 설계했다. 회의 중 나온 "엎드린 사람은 못 잡는다"는 우려는 사진 6장으로 바로 확인했다(모델 6종 모두 6/6). 이후 룸 14개를 모두 찍을 수 없어 3D 디지털 트윈으로 먼저 검증했지만, 엎드린 사람 검출이 48.9%로 실제 사진(6/6)과 달라 하루 만에 중단하고 실제 룸 영상으로 바꿨다. 실제 영상에서는 사전학습 YOLO 16종 중 테이블과 엎드린 사람을 둘 다 잡는 모델이 없었다(YOLO26s 테이블 2%). 우리 룸에서 학습 데이터 727장을 직접 만들어 파인튜닝했고, 1차 학습에서 사람을 잊어버리는 문제(사람 100%→68%)를 겪은 뒤 v2에서 테이블과 엎드린 사람을 모두 100% 잡았다. 사각형 테이블 박스가 통로까지 덮는 문제는 SAM 테두리 +15%와 겹침 기준 10%로 풀었다. 비용은 초기 207만 원, 연 252만 원(학생지원비의 0.02%)으로 정했고, 판정 방식과 도입 효과를 보여 주려고 MVP 관제 대시보드를 만들었다(저녁 식사 시간 룸 활성화율 22%→50%). 최종 발표 뒤에는 당일 받은 피드백을 반영해 프로젝트를 보강했다. 카메라 여러 대 → 판정 → 신호등·기록·현황 화면을 한 번에 돌리는 통합 실행 프로그램을 만들어, 룸 14개를 1초 간격으로 판정할 수 있음을 확인했다(한 바퀴 중앙값 453ms).

1. 연구 질문

총괄을 맡으면서 "실제 룸에서 믿고 쓸 수 있는 판정을 학교가 낼 수 있는 비용으로 만든다"는 목표를 일곱 가지 질문으로 나누고, 각각 실험으로 답했다.

표 1. 연구 질문, 방법, 결론

번호	연구 질문	방법	결론
RQ1	엎드린 사람을 사전학습 YOLO로 잡을 수 있는가	회의 중 위에서 찍은 엎드린 사람 사진 6장, 모델 6종	가능(6종 모두 6/6). 기본 모델 YOLO11s
RQ2	3D 디지털 트윈으로 판정을 검증할 수 있는가	룸 14개 트윈, 가상 학생, 천장 카메라 렌더 112장	불가. 실제 룸 영상으로 전환
RQ3	사전학습 YOLO로 천장 각도의 테이블과 엎드린 사람을 잡을 수 있는가	16종 × 실제 영상, 직접 센 정답으로 채점	불가. 727장으로 파인튜닝 → YOLO26s v2
RQ4	사각형 테이블 박스로 만든 감지 범위의 한계를 어떻게 넘는가	SAM 테두리 + 부풀리기, 겹침 기준 4가지 비교	SAM 테두리 +15%, 겹침 10%
RQ5	학교가 낼 수 있는 최소 비용과 서버 사양은 얼마인가	항목별 비용표, 학교 예산 비교, GPU 실측 환산	초기 207만 원, 연 252만 원(0.02%), 최소 GTX 1060급
RQ6	판정 방식과 도입 효과를 한눈에 보여 줄 수 있는가	실제 영상 22분을 더미 데이터로 쓴 관제 대시보드	저녁 식사 시간 활성화율 22%→50%
RQ7	카메라 여러 대를 한 프로그램으로 실시간 판정할 수 있는가 (발표 후 보강)	통합 실행 프로그램, 영상 복제로 룸 14개 동시 처리	한 바퀴 453ms, 1초 간격 판정 가능

2. 실험 환경과 데이터

표 2. 실험 환경

항목	내용
장치	팀 노트북(RTX 5060 Laptop GPU, 라이젠 7 250). 큰 모델(YOLO26m·l) 학습은 코랩 A100
소프트웨어	Python, Ultralytics 8.4.176, PyTorch 2.11(CUDA 12.8), SAM 2.1 tiny, OpenCV
입력 크기	960 (모든 실험 공통)
시험 데이터	A: 천장 고정 카메라 영상 1차.mp4(22분, 960×540) 중 학습에 쓰지 않은 뒤 20% · B: 라즈베리파이 웹캠 영상(18초, 3명 공부 중, 36장) · 엎드린 사람: 1차.mp4 마지막 7초(84프레임)
학습 데이터	727장. 학습 637장(휴대폰 촬영 144 + 1차.mp4 앞 80% 475 + 엎드린 사람 사진 18), 검증 90장 (휴대폰 37 + 1차.mp4 53)
기간	2026-10-13 ~ 10-23 (기획 10/13, 1차 회의 10/15, 최종 발표 10/22)

3. 문제 정의와 기획 (1주차)

주제는 제조업 비전에서 출발해 출튀 감지를 거쳐 오픈스페이스 알박기 감지로 좁혔다. 융합교육관 스터디룸 14개를 직접 둘러보며 운영 방식을 조사했다. 룸은 예약 없이 선착순으로 운영되고, 학교에는 이미 30분 자리 비움 규칙과 룸 앞 보관함(회색 바구니)이 있었다(그림 1). 그런데 방금 온 학생은 짐 주인이 언제 나갔는지 알 수 없어, 짐만 있는 룸을 두고 그냥 돌아섰다. 그래서 핵심 문제를 "30분 규칙은 있는데, 지켜졌는지 확인할 수 없다"로 정했다. 새 규칙을 만들지 않고, 이미 있는 규칙을 누구나 확인할 수 있게 만드는 것이 목표다.

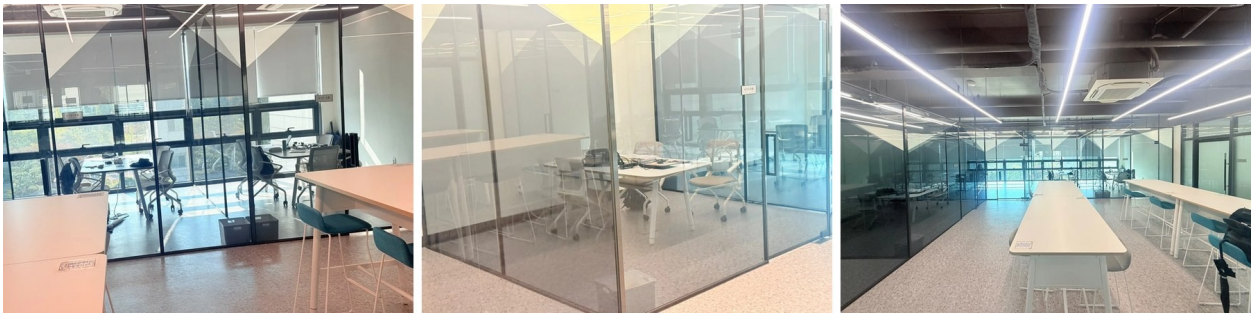
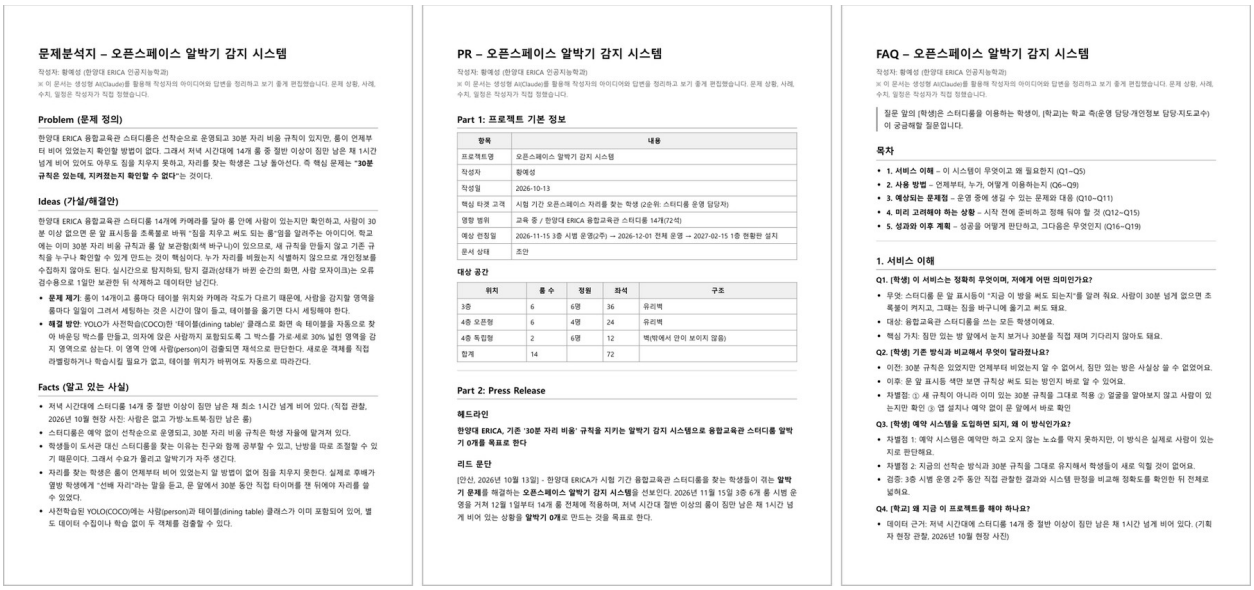


그림 1. 융합교육관 스터디룸 현장 조사 사진. 유리벽으로 나뉜 룸과 오픈형 공간

이 내용을 6 Pager PR/FAQ 양식의 기획서로 정리했다(그림 2). 문제분석지, PR, FAQ 19문항으로 구성했고, FAQ는 학생과 학교 양쪽이 물을 질문으로 나눴다. HKU Find A Seat, 2023년 도서관 좌석 감지 논문, 국내 라즈베리파이 좌석 감지 논문, 대학 도서관 예약 시스템을 조사해 차별점을 네 가지로 정리했다. 좌석이 아니라 룸 단위로 판정하고, 이미 있는 30분 규칙을 그대로 쓰며, 감지 영역을 룸마다 손으로 그리지 않고 자동으로 정하고, 얼굴을 알아보지 않고 사람이 있는지만 본다.



문제분석지 – 오픈스페이스 알박기 감지 시스템

작성자: 황예성 (한양대 ERICA 인공지능학과)
※ 이 문서는 생성형 AI(Claude)를 활용해 작성자의 아이디어와 답변을 정리하고 보기 좋게 편집했습니다. 문제 상황, 사례, 수치, 일정은 작성자가 직접 정했습니다.

Problem (문제 정의)

한양대 ERICA 융합교육관 스터디룸은 선착순으로 운영되고 30분 자리 비움 규칙이 있지만, 룸이 언제부터 비어 있었는지 확인할 방법이 없다. 그래서 저녁 시간대에 14개 룸 중 절반 이상이 점만 남은 채 1시간 넘게 비어 있어도 아무도 짐을 치우지 못하고, 자리를 찾는 학생은 그냥 돌아선다. 즉 핵심 문제는 "30분 규칙은 있는데, 지켜졌는지 확인할 수 없다"는 것이다.

Ideas (가설/해결안)

한양대 ERICA 융합교육관 스터디룸 14개에 카메라를 달아 룸 안에 사람이 있는지만 확인하고, 사람이 30분 이상 없으면 문 앞 표시등을 초록불로 바꿔 "짐을 치우고 써도 되는 룸"임을 알려주는 아이디어. 학교에는 이미 30분 자리 비움 규칙과 룸 앞 보관함(회색 바구니)이 있으므로, 새 규칙을 만들지 않고 기존 규칙을 누구나 확인할 수 있게 만드는 것이 핵심이다. 누가 자리를 비웠는지 식별하지 않으므로 개인정보를 수집하지 않아도 된다. 실시간으로 탐지하되, 탐지 결과(상태가 바뀐 순간의 화면, 사람 모자이크)는 오류 검수용으로 1일만 보관한 뒤 삭제하고 데이터만 남긴다.

- 문제 제기: 룸이 14개이고 룸마다 테이블 위치와 카메라 각도가 다르기 때문에, 사람을 감지할 영역을 룸마다 일일이 그려서 세팅하는 것은 시간이 많이 들고, 테이블을 옮기면 다시 세팅해야 한다.
- 해결 방안: YOLO가 사전학습(COCO)한 '테이블(dining table)' 클래스로 화면 속 테이블을 자동으로 찾아 바운딩 박스를 만들고, 의자에 앉은 사람까지 포함되도록 그 박스를 가로·세로 30% 넓힌 영역을 감지 영역으로 삼는다. 이 영역 안에 사람(person)이 검출되면 재석으로 판단한다. 새로운 객체를 직접 라벨링하거나 학습시킬 필요가 없고, 테이블 위치가 바뀌어도 자동으로 따라간다.

Facts (알고 있는 사실)

- 저녁 시간대에 스터디룸 14개 중 절반 이상이 점만 남은 채 최소 1시간 넘게 비어 있다. (직접 관찰, 2026년 10월 현장 사진: 사람은 없고 가방·노트북·짐만 남은 룸)
- 스터디룸은 예약 없이 선착순으로 운영되고, 30분 자리 비움 규칙은 학생 자율에 맡겨져 있다.
- 학생들이 도서관 대신 스터디룸을 찾는 이유는 친구와 함께 공부할 수 있고, 난방을 따로 조절할 수 있기 때문이다. 그래서 수요가 몰리고 알박기가 자주 생긴다.
- 자리를 찾는 학생은 룸이 언제부터 비어 있었는지 알 방법이 없어 짐을 치우지 못한다. 실제로 후배가 열받 학생에게 "선배 자리"라는 말을 듣고, 문 앞에서 30분 동안 직접 타이머를 쟌 뒤에야 자리를 살 수 있었다.
- 사전학습된 YOLO(COCO)에는 사람(person)과 테이블(dining table) 클래스가 이미 포함되어 있어, 별도 데이터 수집이나 학습 없이 두 객체를 검출할 수 있다.

PR – 오픈스페이스 알박기 감지 시스템

작성자: 황예성 (한양대 ERICA 인공지능학과)
※ 이 문서는 생성형 AI(Claude)를 활용해 작성자의 아이디어와 답변을 정리하고 보기 좋게 편집했습니다. 문제 상황, 사례, 수치, 일정은 작성자가 직접 정했습니다.

Part 1: 프로젝트 기본 정보

항목	내용
프로젝트명	오픈스페이스 알박기 감지 시스템
작성자	황예성
작성일	2026-10-13
핵심 타겟 고객	시험 기간 오픈스페이스 자리를 찾는 학생 (2순위: 스터디룸 운영 담당자)
영향 범위	교육 중 / 한양대 ERICA 융합교육관 스터디룸 14개(72석)
예상 런칭일	2026-11-15 3종 시범 운영(2주) → 2026-12-01 전체 운영 → 2027-02-15 1종 현황판 설치
문서 상태	초안

대상 공간

위치	룸 수	정원	좌석	구조
3층	6	6명	36	유리벽
4층 오픈형	6	4명	24	유리벽
4층 독립형	2	6명	12	벽(밖에서 안이 보이지 않음)
합계	14		72	

Part 2: Press Release

헤드라인

한양대 ERICA, 기존 '30분 자리 비움' 규칙을 지키는 알박기 감지 시스템으로 융합교육관 스터디룸 알박기 0개를 목표로 한다

리드 문단

[안산, 2026년 10월 13일] - 한양대 ERICA가 시험 기간 융합교육관 스터디룸을 찾는 학생들이 겪는 알박기 문제를 해결하는 오픈스페이스 알박기 감지 시스템을 선보인다. 2026년 11월 15일 3종 6개 룸 시범 운영을 거쳐 12월 1일부터 14개 룸 전체에 적용하며, 저녁 시간대 절반 이상의 룸이 점만 남은 채 1시간 넘게 비어 있는 상황을 알박기 0개로 만드는 것을 목표로 한다.

FAQ – 오픈스페이스 알박기 감지 시스템

작성자: 황예성 (한양대 ERICA 인공지능학과)
※ 이 문서는 생성형 AI(Claude)를 활용해 작성자의 아이디어와 답변을 정리하고 보기 좋게 편집했습니다. 문제 상황, 사례, 수치, 일정은 작성자가 직접 정했습니다.

> 질문 앞의 [학생]은 스터디룸을 이용하는 학생이, [학교]는 학교 측(운영 담당·개인정보 담당·지도교수)이 궁금해할 질문입니다.

목차

- 1. 서비스 이해 – 이 시스템이 무엇이고 왜 필요한지 (Q1~Q5)
- 2. 사용 방법 – 언제부터, 누가, 어떻게 이용하는지 (Q6~Q9)
- 3. 예상되는 문제점 – 운영 중에 생길 수 있는 문제와 대응 (Q10~Q11)
- 4. 미리 고려해야 하는 상황 – 시작 전에 준비하고 정해 둬야 할 것 (Q12~Q15)
- 5. 성과와 이후 계획 – 성공을 어떻게 판단하고, 그다음은 무엇인지 (Q16~Q19)

1. 서비스 이해

Q1. [학생] 이 서비스는 정확히 무엇이며, 저에게 어떤 의미인가요?

- 무엇: 스터디룸 문 앞 표시등이 "지금 이 방을 써도 되는지"를 알려 줘요. 사람이 30분 넘게 없으면 초록불이 켜지고, 그때는 짐을 바구니에 옮기고 써도 돼요.
- 대상: 융합교육관 스터디룸을 쓰는 모든 학생이에요.
- 핵심 가치: 짐만 있는 방 앞에서 눈치 보거나 30분을 직접 재며 기다리지 않아도 돼요.

Q2. [학생] 기존 방식과 비교해서 무엇이 달라졌나요?

- 이전: 30분 규칙은 있었지만 언제부터 비었는지 알 수 없어서, 짐만 있는 방은 사실상 쓸 수 없었어요.
- 이후: 문 앞 표시등 색만 보면 규칙상 써도 되는 방인지 바로 알 수 있어요.
- 차별점: ① 새 규칙이 아니라 이미 있는 30분 규칙을 그대로 적용 ② 얼굴을 알아보지 않고 사람이 있는지만 확인 ③ 앱 설치나 예약 없이 문 앞에서 바로 확인

Q3. [학생] 예약 시스템을 도입하면 되지, 왜 이 방식인가요?

- 차별점 1: 예약 시스템은 예약만 하고 오지 않는 노쇼를 막지 못하지만, 이 방식은 실제로 사람이 있는지로 판단해요.
- 차별점 2: 지금의 선착순 방식과 30분 규칙을 그대로 유지해서 학생들이 새로 익힐 것이 없어요.
- 검증: 3종 시범 운영 2주 동안 직접 관찰한 결과와 시스템 판정을 비교해 정확도를 확인한 뒤 전체로 넓혀요.

Q4. [학교] 왜 지금 이 프로젝트를 해야 하나요?

- 데이터 근거: 저녁 시간대에 스터디룸 14개 중 절반 이상이 점만 남은 채 1시간 넘게 비어 있다. (기획자 현장 관찰, 2026년 10월 현장 사진)

그림 2. 1주차 기획서 첫 쪽. 왼쪽부터 문제분석지, PR, FAQ

판정 방식도 이때 설계했다. YOLO11s로 테이블과 사람을 찾고, 테이블 박스를 가로·세로 30% 넓힌 감지 영역 안에 사람이 있으면 재석으로 본다. 사람이 10초 넘게 안 보여야 부재로 보는 가림 보정과 30분 기준 3색 규칙을 정했고, 상태가 바뀐 순간의 화면만 사람을 모자이크해 1일 보관하는 방식도 이때 정했다. 이 판정 코드(MVP)와 규칙 테스트 6개를 만들어 이후 실험의 기준으로 썼다. 아래 표는 1주차 설계가 프로젝트를 거치며 어떻게 바뀌었는지 정리한 것이다.

표 3. 1주차 설계와 최종 결정

항목	1주차 설계	최종 (바뀐 이유)
검출 모델	사전학습 YOLO11s	파인튜닝 YOLO26s v2 (천장 각도의 테이블을 못 잡아 6절)
감지 영역	테이블 박스를 가로·세로 30% 넓힘	SAM 테이블 테두리 +15%, 겹침 10% (박스가 통로까지 덮어 7절)
재석·부재	10초 넘게 안 보이면 부재	같음, 2초 연속 보여야 재석
신호등	빨강 사용 중, 노랑 30분 미만, 초록 30분 이상	빈 책상(사람도 짐도 없음)은 바로 초록 추가 (7절)
판정 장치	룸별 라즈베리파이 추론 검토	교내 PC 1대 (라즈베리파이 1장 2.8초·81.3°C)
보관	상태가 바뀐 순간만 모자이크해 1일 보관	같음

4. 엎드린 사람 검증과 기본 모델 선정 (RQ1)

1차 회의(10/15)에서 "책상에 엎드린 사람은 사전학습 YOLO가 잘 못 잡을 것"이라는 의견이 나왔다. 엎드린 사람을 놓치면 공부 중인 룸에 초록불이 켜지므로 그냥 넘길 수 없었다. 그래서 회의 중에 스터디룸에서 엎드린 모습을 위에서 광각으로 6장 찍어, 모델 6종을 바로 돌려 봤다(그림 3).

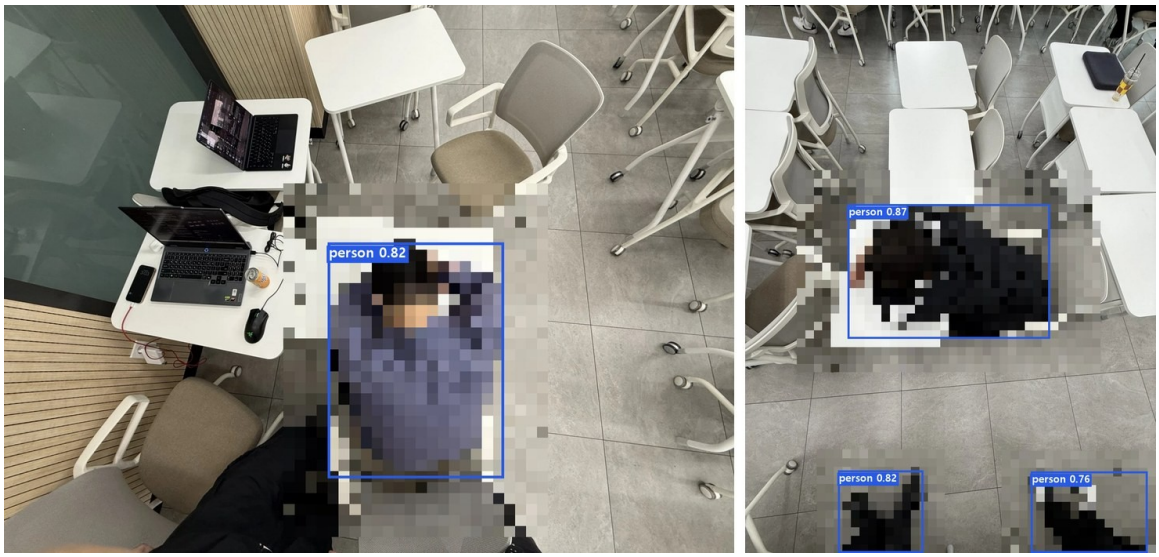


그림 3. 회의 중 찍은 엎드린 사람 사진과 YOLO11s 검출 결과(기준 0.35). 사람은 모자이크했다

표 4. 모델 6종의 엎드린 사람 검출 (사진 6장, 기준 0.35)

모델	평균 신뢰도	검출	CPU 1장
YOLO11x	0.95	6/6	602ms
YOLO11s (선택)	0.85	6/6	94ms
YOLOv8s	0.83	6/6	—
YOLOv8n	0.71	6/6	—
YOLO11n	0.70	6/6	48ms
YOLO26n	0.57	6/6	51ms

우려와 달리 6종 모두 6장을 전부 잡았다. 그래서 정확도와 속도의 균형으로 YOLO11s를 기본 모델로 정했다. YOLO11x는 평균 신뢰도가 0.95로 가장 높지만 CPU에서 1장에 602ms가 걸리고 파일도 109MB라, 0.85에 94ms인 YOLO11s가 맞았다. 실제 스터디룸 영상에서도 공용 테이블이 0.63~0.71로 잡혀, 이때는 사전학습 모델로 충분하다고 판단했다. 이 판단은 천장 고정 카메라 영상을 돌려 보면서 뒤집혔다(6절).

5. 3D 디지털 트윈 검증과 중단 (RQ2)

룸 14개를 실제로 다 찍을 수 없어서, 먼저 융합교육관 스터디룸 14개를 3D 디지털 트윈으로 만들었다. 가상 학생을 앉히거나 엎드리게 하고, 룸마다 천장 카메라 화면을 렌더링해 YOLO 서버로 판정했다(그림 4). 렌더 112장을 YOLO11s, 기준 0.35로 쟀다.



그림 4. 스터디룸 14개 디지털 트윈. 왼쪽은 룸을 고르면 천장 카메라 화면과 판정을 보여 주는 화면, 오른쪽은 측정에 쓴 렌더 예시다. 사람은 모두 가상 인물이다

표 5. 3D 렌더와 실제 사진의 검출 결과

자세·판정	3D 렌더 (112장)	실제 사진
앉아서 공부	98.2% (160/163)	—
엎드려 잠	48.9% (43/88)	100% (6/6, 평균 신뢰도 0.85)
룸 판정	92.0% (사람 있는 룸 75/84, 빈 룸 28/28)	—

앉은 사람과 룸 판정만 보면 쓸 만해 보였지만, 세 가지 이유로 하루 만에 중단했다.

- **점수가 그래픽을 쟀다:** 가장 중요한 엎드린 사람이 3D에서는 48.9%였지만, 같은 모델이 실제 사진에서는 6장을 모두 잡았다. 가상 인물의 모양과 질감이 실제 사람과 달라, 점수가 모델 성능이 아니라 그래픽의 한계를 재고 있었다.
- **환경마다 결과가 달랐다:** GPU 없이 렌더링하면 4명이 있는 룸을 0명으로 판정했다. 이런 결과로는 결정을 내릴 수 없다.
- **실제로 찍는 편이 빨랐다:** 알고 싶은 것은 실제 룸의 천장 각도·조명·테이블에서의 성능인데, 가상 화면을 실제에 맞추는 시간이 실제 룸을 한 번 찍는 시간보다 컸다.

그래서 실제 룸 천장에 카메라를 달아 22분을 찍고, 이 영상으로 검증하는 방식으로 바꿨다.

6. 사전학습 모델 비교와 파인튜닝 (RQ3)

실제 영상을 돌려 보니 더 큰 문제가 나왔다. 사전학습 YOLO는 책상을 식탁(dining table)으로만 배워, 천장에서 내려다본 우리 테이블을 거의 찾지 못했다(YOLO26s 2%). 테이블을 못 찾으면 감지 범위를 만들 수 없다.

모델 16종 비교. 더 큰 모델이면 잡을지 보려고, 라즈베리파이 웹캠 영상에서 0.5초마다 뽑은 36장으로 크기별 11종(v8·11·12·26)과 구조가 다른 5종(v5u, v9, v10, RT-DETR, YOLO-World)을 같은 조건(입력 960, 기준 0.25)으로 돌렸다. 장면에 실제로 있는 개수(사람 3, 노트북 3, 휴대폰 2, 마우스 1, 가방 1, 테이블 1)를 직접 세어 정답으로 쓰고, 엎드린 사람은 1차.mp4 마지막 7초(84프레임)를 기준 0.35로 따로 채점했다.

표 6. 사전학습 16종 중 대표 8종 (웹캠 36장, 엎드린 사람 84프레임)

모델	물건 5종	테이블	엎드린 사람	파라미터	CPU 1장
yolov8n	62%	8%	56.0%	3.2M	67ms
yolo11s	72%	0%	6.0%	9.5M	142ms
yolo11l	100%	100%	31.0%	25.4M	493ms
yolov9s	85%	92%	2.4%	7.3M	181ms
yolov10s	100%	8%	97.6%	8.1M	131ms
yolo26n	63%	3%	65.5%	2.6M	69ms
yolo26s	96%	0%	92.9%	10.0M	155ms
rtdetr-l	100%	72%	91.7%	33.0M	1,055ms

사람은 모든 모델이 잘 잡았지만, 테이블과 엎드린 사람을 둘 다 잡는 모델은 없었다. 테이블을 잡는 yolo11l은 엎드린 사람이 31%, 엎드린 사람을 잡는 yolov10s는 테이블이 8%였다. 모델 크기가 아니라 데이터의 문제라고 보고, 사전학습 모델 중에서 고르는 대신 우리 룸 영상으로 파인튜닝하기로 했다.

학습 데이터 727장. 테이블 라벨은 테이블을 잘 잡던 4개 모델(yolo11l, yolo11m, yolov9s, rtdetr-l)의 박스 중 가장 확실한 것을 후보로 쓰고, 휴대폰 영상 181장은 전부 눈으로 검수했다. 옆 룸의 작은 테이블을 잡은 3장은 본 테이블로 바꾸고, 테이블 절반만 잡은 1장은 손으로 다시 그렸으며, 의자까지 포함해 너무 커진 4장은 되돌렸다. 사람·물건 라벨은 엎드린 사람을 가장 잘 잡은 yolov10s로 자동 라벨링했다. 클래스 번호는 COCO 그대로(0=사람, 60=테이블) 둬서 기존 코드에 모델 파일만 바꿔 끼우면 되게 했고, 1차.mp4 뒤 20%와 웹캠 영상은 학습에 넣지 않고 시험에만 썼다.

표 7. 학습 데이터 구성

출처	장수	테이블 라벨
휴대폰으로 테이블 주변을 돌며 찍은 영상 18초	181장 (3프레임마다)	4개 모델 후보 → 181장 전부 눈으로 검수, 틀린 박스는 손으로 수정
천장 고정 카메라 영상 1차.mp4 앞 80%	528장 (2초마다)	카메라가 고정이라 정답 박스 하나를 모든 프레임에 사용
위에서 찍은 엎드린 사람 사진	6장 × 3번 복사	4개 모델 후보 + 눈으로 검수

1차 학습의 실패와 v2. 1차 학습은 테이블 위주(휴대폰 영상만)라, 테이블은 100%가 됐지만 사람과 가방을 잊어버렸다(웹캠 사람 100%→68%, 가방 89%→14%, 엎드린 사람 6%→0%). 새 데이터에 맞추느라 원래 알던 것을 덮어쓴 것으로 보고, 1차.mp4 프레임과 엎드린 사람 사진을 함께 넣고 학습률을 낮춰(AdamW, lr0 0.001, 25에포크) v2를 학습했다. 백본을 고정한 학습과도 비교했지만 가방이 25%에 그쳐 전체 학습을 택했다.

표 8. 파인튜닝 전후 (학습에 쓰지 않은 영상 기준. A: 1차.mp4 뒤 20%, B: 웹캠)

모델	테이블 A	테이블 B	엎드린 사람	앉은 사람	파라미터
YOLO11s 원본	95%	0%	6%	100%	9.5M
YOLO26s 원본	2%	0%	92.9%	100%	10.0M
YOLO11s v2	100%	100%	98.8%	100%	9.5M
YOLO26s v2 (최종)	100%	100%	100%	100%	10.0M
YOLO26m v2	100%	100%	100%	100%	21.9M

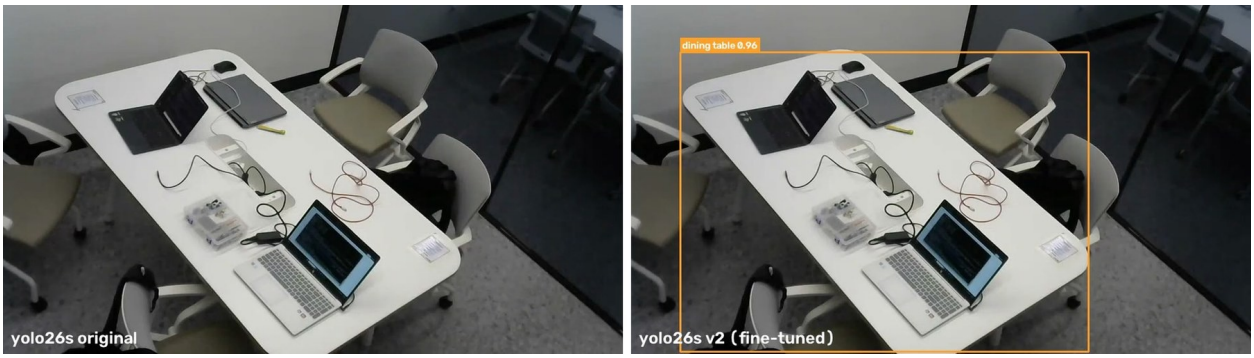


그림 5. 짐만 남은 빈 자리. 원본 YOLO26s(왼쪽)는 테이블을 잡지 못했고, 파인튜닝한 v2(오른쪽)는 신뢰도 0.96으로 잡았다

최종 모델은 파라미터 대비 성능으로 골랐다(그림 6). YOLO26n은 엎드린 사람이 65.5%(파인튜닝 전)라 사람이 있는데도 초록불이 켜질 위험이 있고, GPU에서는 s와 속도 차이도 거의 없었다(17.9ms 대 18.1ms). YOLO26m·l은 정확도가 s와 같으면서 CPU에서 2.7~3.4배 느렸다. 그래서 최종 모델을 YOLO26s v2로 정했다.

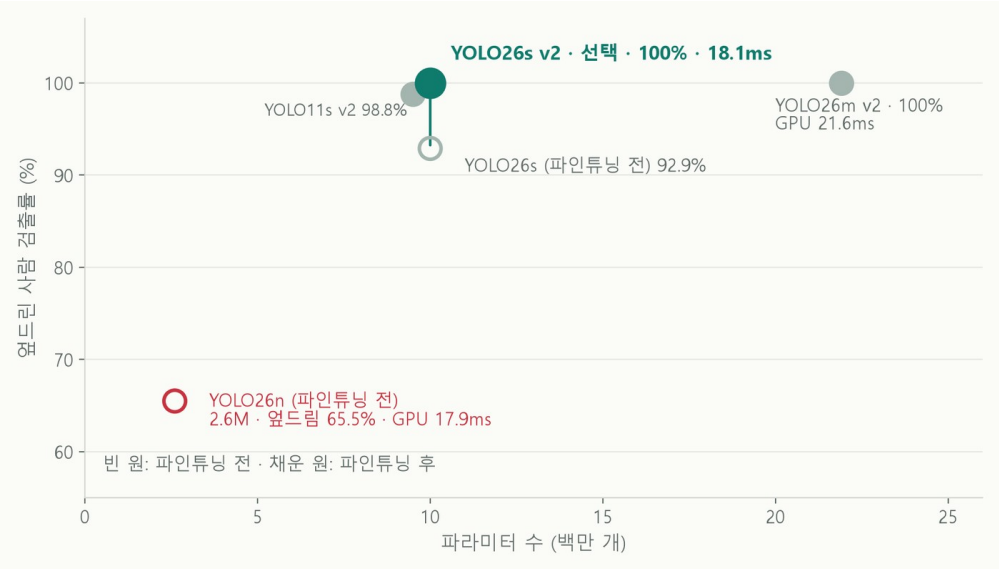


그림 6. 파라미터 수와 엎드린 사람 검출률. 빈 원은 파인튜닝 전, 채운 원은 파인튜닝 후다

7. 감지 범위: 테이블 박스에서 SAM 테두리로 (RQ4)

테이블을 잡게 된 뒤에도 문제가 남았다. 처음에는 YOLO 테이블 박스를 가로·세로 30% 넓혀 감지 범위로 썼는데, 박스는 사각형이라 비스듬히 놓인 테이블에서는 옆 통로까지 덮어 지나가는 사람도 재석으로 셌다. 박스를 넓히거나 줄이는 것만으로는 통로와 테이블 끝자리를 동시에 맞출 수 없어, 기존 YOLO 인식 범위의 한계라고 판단했다.

그래서 감지 범위를 세 단계로 만들었다(그림 7). ① 빈 룸에서 YOLO26s v2가 테이블 박스를 찾고, ② 그 박스를 SAM에 넣어 테이블 테두리만 딴 뒤, ③ 테두리를 짧은 변의 15%만큼 부풀린다. 사람에 가려지지 않은 빈 룸에서 한 번 따고, 테이블 위치가 달라진 상태가 30초 넘게 이어지고 사람이 없을 때만 다시 딴다.



그림 7. SAM 감지 범위 3단계. ① YOLO 테이블 박스 → ② SAM 테이블 테두리 → ③ 짧은 변의 15%만큼 부풀린 감지 범위

테이블 끝에 앉은 사람까지 잡으려고 범위를 더 넓히자 유리벽 너머 옆방 사람까지 잡혔다. 그래서 범위는 +15%로 두고, 사람 박스가 범위와 겹쳐야 하는 비율(겹침 기준)을 낮췄다. 1차.mp4에서 사람 555번을 기준별로 비교한 결과다.

표 9. 겹침 기준 비교 (1차.mp4, 사람 555번, 1초 단위 378초)

겹침 기준	사람별로 재석으로 센 비율	사람이 있는데 룸을 비었다고 본 순간
25% (처음)	95.3%	1초
15%	96.9%	1초 (마지막 사람이 나가는 순간)
10% (최종)	97.5%	0초
1%	99.5%	0초

1%까지 낮추면 감지 범위에 살짝 닿기만 해도 재석이 되어, 짐만 남은 룸 앞 통로에 누가 서 있으면 부재 타이머가 계속 처음으로 돌아간다. 그래서 시험한 값 중 룸 판정이 틀린 순간이 없는 가장 높은 기준인 10%로 정했다.

빈 책상 규칙. 노랑은 "짐 주인이 돌아올 수 있으니 30분 기다린다"는 뜻인데, 짐까지 챙겨 나간 빈 책상도 30분을 기다리는 것은 낭비였다. 그래서 사람도 짐도 10초 연속 보이지 않으면 노랑을 거치지 않고 바로 초록으로 바꾸도록 판정 규칙을 고쳤다. 짐은 감지 범위 안의 가방·노트북·책 등 11종으로 보고, 가방을 놓치면 짐 있는 책상이 빈 책상으로 보이므로 가방류는 신뢰도 기준을 0.10으로 낮췄다.

8. 비용과 서버 사양 (RQ5)

교수님 피드백("비용이 정말 최소인지 보여 줘라, 오픈소스도 공짜가 아니다")에 맞춰, 학교에 파는 B2B 기준으로 비용을 항목별로 계산했다(그림 8). 처음 계산에는 PoE 스위치, 업체 시공, 1층 현황판 모니터가 들어 있어 학교가 내기엔 컸다. 와이파이 카메라(Tapo C210)로 바꿔 PoE와 시공을 빼고, 현황판은 고도화 단계로 미뤘다. Tapo C210은 앱 전용 홈캠과 달리 RTSP로 영상을 직접 받을 수 있어, 영상이 제조사 클라우드를 거치지 않는다.



그림 8. 시스템 컨텍스트 다이어그램. 고객은 학교 운영 담당, 사용자는 학생이며, 판정은 교내 PC 1대에서 한다

표 10. 초기 비용 (룸 14개)

항목	계산	금액
카메라 (Tapo C210)	14대 × 45,900원	64만 원
판정 서버 PC (DB 포함)	1대	50만 원
문 앞 신호등	14개 × 35,000원	49만 원
전기 공사	전체	30만 원
카메라 고정 부자재	14곳 × 10,000원	14만 원
PoE 스위치 · 1층 현황판	와이파이 카메라라 불필요 · 고도화 단계로 미룸	0원
합계	룸당 약 15만 원	207만 원

운영비는 룸 단위 구독으로 정했다. 룸당 월 1.5만 원(14개 룸이면 월 21만 원, 연 252만 원)이고, 점검·고장 수리(출장비 제외)·재학습 인건비를 넣었다. 룸 단위라 일부 룸부터 도입할 수 있다. 한양대 2026학년도 교비회계 자금예산서(ERICA)의 학생지원비 103.6억 원과 비교하면 연 구독료는 0.02%, 초기 비용을 더한 첫해도 0.04%다.

서버 최소 사양은 실측으로 정했다. 룸 14개를 1초에 1장씩 판정하려면 1장을 71ms 안에 처리해야 한다. 팀 노트북에서 YOLO26s v2(입력 960)를 30번 돌린 중앙값으로 재고, GPU 연산 성능(TFLOPS) 비율로 학교 PC급 GPU를 환산했다.

표 11. 1장 처리 시간과 서버 사양

방식	1장 처리	초당 처리	71ms 기준
CPU만 (라이젠 7 250)	135ms	7.4장	부족
CPU + GPU (RTX 5060 노트북, 실측)	18.1ms	55장	3.9배 여유
GTX 1070 (6.5 TFLOPS, 환산)	약 42ms	약 24장	통과
GTX 1060 (4.4 TFLOPS, 환산)	약 62ms	약 16장	통과, 여유 적음

그래서 최소 GTX 1060급, 권장 GTX 1070급으로 정했다. CPU는 카메라 14대의 영상을 받아 푸는 일을 맡으므로 6코어 라이젠 5 5600급, 램 16GB로 잡았다(PC 약 50만 원).

9. MVP 관제 대시보드 (RQ6)

판정 방식과 도입 효과를 말로 설명하면 이해하기 어려웠다. 눈으로 보는 것이 더 쉽다고 판단해, 우리 학습 데이터(실제 룸 천장 카메라 영상 22분)를 더미 데이터로 삼아 14개 룸을 가상으로 운영하는 관리자 대시보드를 만들었다. 룸 지도와 상태, 선택한 룸의 카메라 화면과 판정 근거, 부재 기록 조회, 통계·KPI, 1분 단위 데이터 기록(CSV), 1층 현황판 미리보기, 도입 전·후 비교를 한 화면에서 본다(그림 9).



그림 9. MVP 관제 대시보드, 도입 전 18:30. 14개 룸 중 사용 중 2개, 30분 넘게 짐만 있는 룸 8개, 룸 활성화율 14%

표 12. 도입 전 → 도입 후 (저녁 식사 시간, 같은 날 시나리오)

지표	도입 전 (아무도 안 치움)	도입 후 (초록불 → 짐 치우고 사용)
룸 활성화율 (추천 시나리오)	22%	50%
30분 넘게 짐만 있는 룸 평균	6.4개	0.6개
초록불이 켜진 뒤 다시 사용까지 평균	47분	2분
룸 활성화율 (무작위 시나리오 300개 평균)	26%	51%

배포한 링크에서는 로그인 없이 관리자 화면이 바로 열리고, 17:00부터 재생되며 Primary KPI는 저녁 5~10시 평균이다. 다만 시뮬레이션이라 실제 운영 수치가 아니고, 화면의 감지 범위는 SAM 도입 전 방식(테이블 박스 +30%)으로 그려져 있다.

10. 프로젝트 보강: 통합 실행 프로그램 (RQ7)

이 절은 최종 발표(10/22) 당일 받은 피드백을 반영해, 발표 뒤에 보강한 내용이다. 발표 시점에는 판정 코드, 모델, 대시보드가 따로 있어서, 카메라 영상을 받아 판정하고 신호등까지 보내는 프로그램은 없었다. 그래서 카메라(또는 영상) 여러 대 → 판정 → 신호등·기록·실시간 현황 화면을 한 번에 돌리는 알박기_실행.py를 만들었다.

- **최신 화면만 받기:** 룸마다 카메라에서 가장 최근 화면 한 장만 들고 있어, 판정이 느려도 밀린 화면이 쌓이지 않는다.
- **1초마다 모든 룸 판정:** YOLO·SAM 모델은 하나를 모든 룸이 같이 쓰고, 감지 범위와 타이머는 룸마다 따로 둔다.
- **색이 바뀔 때만 알림:** 신호등 주소로 색을 보내고, 사람을 모자이크한 스냅샷을 1일 보관하며, 모든 판정을 CSV로 남긴다.
- **현황 화면:** MVP 대시보드와 같은 디자인에 실제 판정 결과를 1초마다 띄운다. 선택한 룸의 카메라는 판정과 따로 초당 15장으로 실시간 전송하고, 박스가 최대 1초 늦게 따라오는 만큼 모자이크를 사람 박스보다 25% 넓게 씌운다(그림 10).
- **시험용 초록불 시간:** 시작할 때 초록불 시간을 00:01:30(또는 000130) 형식으로 입력받아, 30분을 기다리지 않고 시험한다.

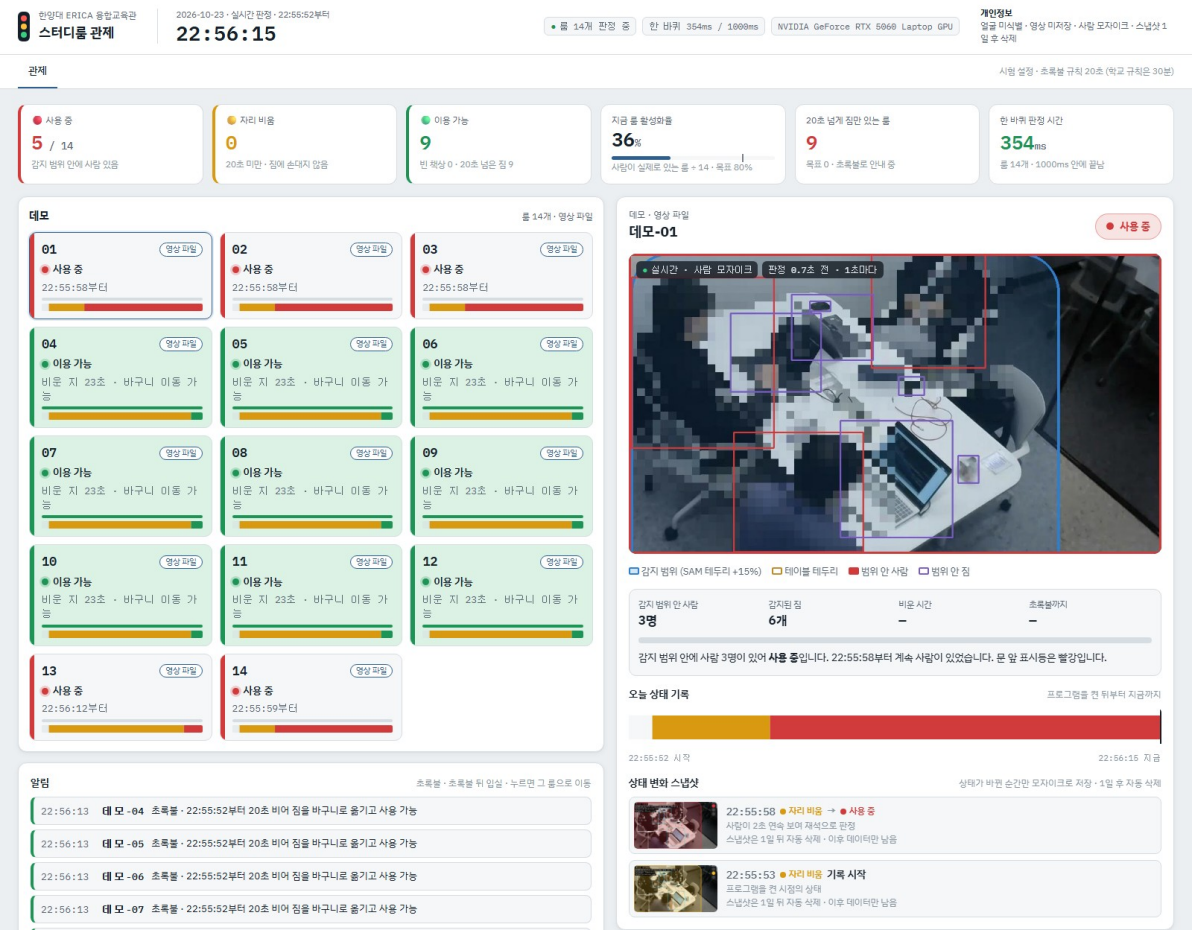


그림 10. 통합 실행 현황 화면. 룸 14개(영상 복제)를 1초마다 판정하고, 선택한 룸의 카메라는 실시간으로 보여 준다 (초록불 시간 20초로 시험, 사람은 모자이크)

교수님이 요청한 병렬 테스트로, 1차.mp4를 룸 14개처럼 복제해(룸마다 다른 위치부터 재생) 60초 동안 1초마다 판정했다.

표 13. 룸 14개 동시 처리 (RTX 5060 Laptop GPU, 입력 960)

항목	값
한 바퀴(14개 룸) 처리 시간 중앙값	453ms
p95	708ms
룸 1개당	32.4ms
가장 긴 바퀴	5,643ms (여러 룸이 동시에 SAM으로 감지 범위를 새로 딴 바퀴)
1초 간격 실시간 판정	가능 (p95가 1,000ms보다 짧음)

판정이 실제 카메라에서 어떻게 보이는지 확인하려고 웹캠용 성능 확인 창(성능확인.py)도 만들었다(그림 11). 화면은 카메라 그대로 흐르고 판정은 1초마다 하며, 화면 fps, 판정 시간, 이 컴퓨터로 1초 간격 판정이 가능한 룸 수를 보여 준다. 여기서 예상하지 못한 결과가 나왔다. 파인튜닝 v2가 정면 웹캠 앞 사람을 거의 찾지 못한 것이다.

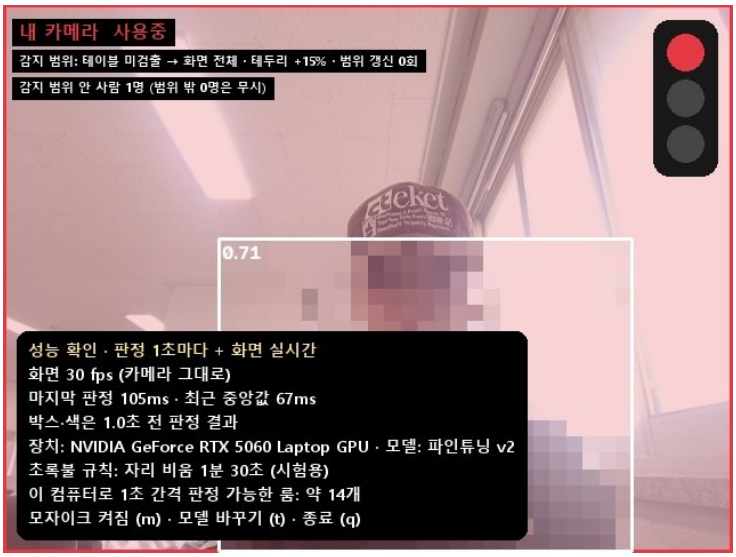


그림 11. 성능 확인 창(사람 모자이크). 왼쪽 아래에 판정 시간, 모델, 초록불 시간, 가능한 룸 수가 나온다

표 14. 정면 웹캠 사람 검출 (같은 화면 14장, 기준 0.35)

모델	검출	최고 신뢰도
파인튜닝 v2	14장 중 2장	대부분 0.06 이하
기본 YOLO26s	14장 중 14장	0.91~0.93

v2는 천장에서 내려다본 스터디룸 사진으로만 학습해, 학습한 각도에는 강해진 대신 정면 각도에는 약해졌다. 실제 설치는 천장 카메라라 운영에는 v2를 쓰고, 성능 확인 창은 웹캠이면 기본 YOLO26s로 시작하되 t 키로 두 모델을 바꿔 비교할 수 있게 했다.

성능 확인 창의 "가능한 룸 수"(14~17개)가 발표의 "초당 55장"보다 낮게 나온 이유도 측정 범위별로 나눠 확인했다.

표 15. 측정 범위별 룸 1개 처리 시간

측정	룸 1개	1초에 가능한 룸	들어간 일
발표 (서버 사양 실측)	18.1ms	약 55개	YOLO 추론만
판정만, 창 없음 (10/23, 배터리 전원)	29.4ms	약 34개	YOLO + 감지 범위·짐 판정
룸 14개 동시 처리	30~32ms	약 30개	판정 + 화면 복사 + 기록
성능 확인 창 (10/23, 배터리 전원)	57~71ms	약 14~17개	판정 + 화면을 초당 27장 그리기

성능 확인 창은 판정과 화면 그리기를 한 프로그램에서 같이 해서 가장 느렸고, 10/23 측정은 노트북이 배터리 전원이라 YOLO만 돌려도 27ms가 나왔다. 실제 서버는 창 없이 판정만 하므로, 룸 14개를 1초 안에 판정할 수 있다는 결론은 가장 불리한 조건에서도 바뀌지 않는다. 다만 실제 카메라 14대(RTSP)와 GTX 1060급 실측은 아직 하지 않았다.

11. 발표 자료와 프로젝트 검수

발표 자료(17장)를 만들고 발표했다. 구성은 문제 → 타깃과 판단 기준 → 선택(카메라·판정 위치·모델·데이터) → 감지 범위와 판정 → 서버 사양 → 비용 → 개인정보 → MVP → 추후 계획 순서로, 무엇을 시도했고 왜 그렇게 정했는지가 이어지게 했다(그림 12).

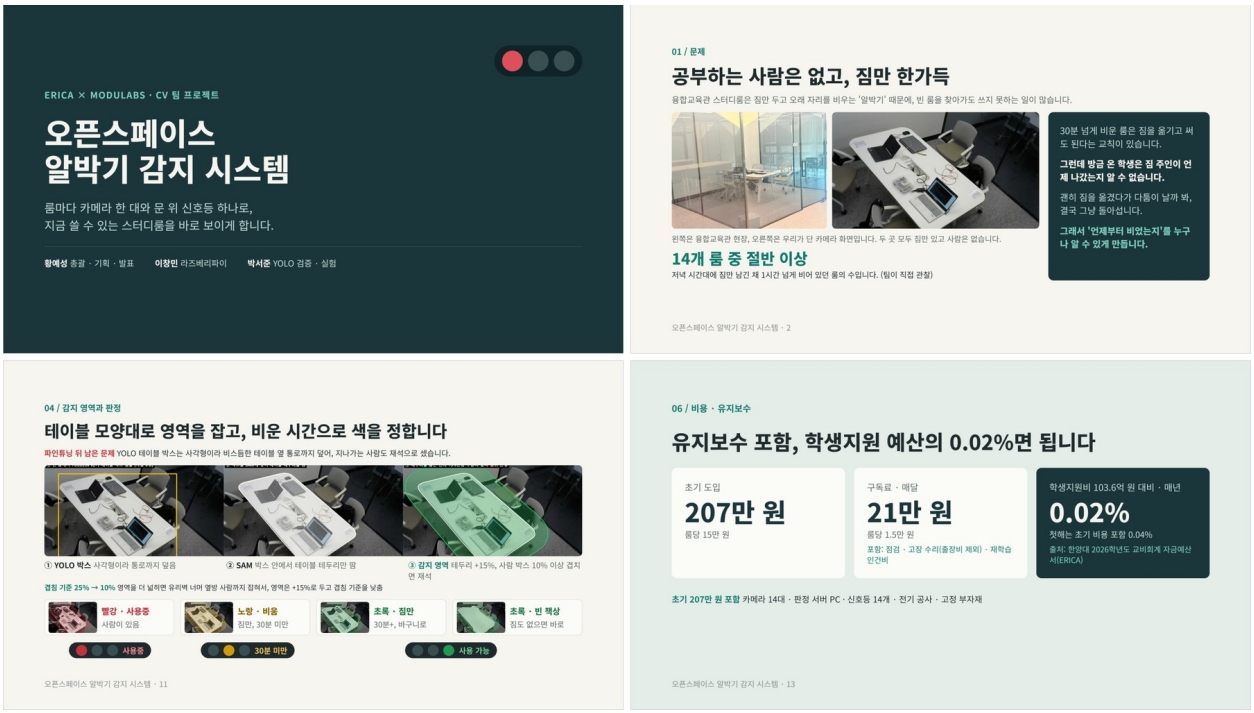


그림 12. 발표 자료 일부. 표지, 문제, 감지 범위와 판정, 비용

- **팀원 피드백 반영:** 팀원 수정본의 생성형 데이터 슬라이드와 문제·개인정보 슬라이드 수정안을 합치고, 리허설에서 드러난 흐름과 문구 문제(신호등 배치, 슬라이드 제목 등)를 고쳤다.
- **발표문:** 리허설 녹음 두 번을 받아 적어 발표문으로 다듬고 발표자 노트에 넣었다.
- **시범 운영 불합격 기준:** 300번 관찰해 잘못된 초록이 0건이면 실제 오류율이 1% 이하라고 95% 신뢰로 말할 수 있다(3의 규칙). 이를 근거로 잘못된 초록 1건 이상 또는 빈 룸 놓침 2% 초과를 불합격으로 정했다.
- **수치 검수:** 슬라이드·보고서·코드에 흩어진 수치를 한 표(수치표)로 모아 서로 다른 값을 찾아 고쳤다. 예를 들어 대시보드 규칙 카드에 남아 있던 옛 감지 범위(+30%)를 SAM +15%와 빈 책상 규칙으로 바로잡았다.
- **저장소 정리:** 팀 저장소 10_Project를 단계별 폴더(00_최종결과물 ~ 07_통합실행)로 재구성하고, 기획 문서(PR/FAQ, 6 Pager) 최종본에 추가·삭제·변경을 표시했다. 공개 저장소에 올리는 사람 이미지는 모두 모자이크했다.

12. 결론

이번 작업의 기여는 여섯 가지로 정리된다.

- **문제 정의와 설계:** 현장 조사로 "30분 규칙은 있는데 지켜졌는지 확인할 수 없다"는 문제를 정하고, 기획서와 판정 규칙(감지 영역, 가림 보정, 3색)을 세웠다. 엎드린 사람 우려는 회의 중 사진 6장으로 바로 확인했다.

- **검증 방법 바로잡기:** 3D 시뮬레이션의 점수가 모델이 아니라 그래픽의 한계를 잰다는 것을 확인하고, 하루 만에 실제 영상 검증으로 바꿨다.
- **데이터로 푼 테이블 문제:** 모델 크기가 아니라 데이터 문제임을 16종 비교로 확인하고, 727장 파인튜닝으로 테이블 2%→100%, 엎드린 사람 92.9%→100%를 얻었다. 1차 학습에서 사람을 잊은 문제도 데이터 구성과 학습률로 풀었다.
- **모양대로 잡는 감지 범위:** SAM 테두리 +15%와 겹침 기준 10%로 통로·옆방 오탐을 막으면서, 사람이 있는데 룸을 비었다고 보는 순간을 없앴다.
- **학교가 낼 수 있는 비용:** 초기 207만 원, 연 252만 원(학생지원비의 0.02%), 최소 GTX 1060급 서버로 정했다.
- **눈으로 확인하는 결과물:** MVP 대시보드로 도입 효과(저녁 식사 시간 활성화율 22%→50%)를 보여 주고, 발표 후 보강한 통합 실행 프로그램으로 룸 14개를 1초 간격으로 판정할 수 있음을 확인했다.

이창민 개인 연구 보고: 라즈베리파이 카메라 송출과 온디바이스 추론 평가

담당: 라즈베리파이. 카메라 영상 송출, PC 수신 추론 연동, 라즈베리파이 안에서 추론할 때의 성능 평가를 맡았다. 코드와 측정 기록은 GitHub 10_Project/06_라즈베리파이(https://github.com/gosu1/ERICA_X_MODULABS_CV/tree/main/10_Project/06_라즈베리파이)에 있다.

요약

라즈베리파이 4B와 USB 웹캠(Logitech C270)으로 카메라 영상을 PC로 보내는 송출 모듈을 만들고, PC에서 받은 영상으로 YOLO를 돌리는 테스트 스크립트와 수신 스트림 녹화 기능을 만들었다. 2주차에는 송출 방식을 HTTP/MJPEG에서 H.264 인코딩 기반 RTSP로 바꿨고, 이 스트림은 팀의 실시간 인식 화면이 받아 초당 17.8장을 표시했다. 또 최종 모델 YOLO26s v2를 라즈베리파이 안에서 직접 돌리는 벤치마크를 만들어 1차 영상 659장으로 쟀다. 정확도는 서버와 같았지만 1장에 2,831.8ms(초당 0.353장)가 걸리고 최고 81.3°C까지 올라, 팀은 룸별 라즈베리파이 추론을 제외하고 교내 PC 1대에서 판정하기로 했다.

1. 연구 질문

1차 회의에서 검토한 "룸마다 라즈베리파이를 두고 그 안에서 추론하는 안"을 판단하려고 세 가지 질문을 세웠다.

표 1. 연구 질문, 방법, 결론

번호	연구 질문	방법	결론
RQ1	라즈베리파이로 카메라 영상을 PC에 실시간으로 보낼 수 있는가	USB 웹캠 → HTTP/MJPEG 송출, 이후 H.264 RTSP 송출로 전환	가능. RTSP 1280×720·15fps
RQ2	받은 영상으로 PC에서 판정을 돌릴 수 있는가	수신 영상 YOLO 추론 테스트 스크립트, 수신 스트림 녹화	가능. 녹화 기능으로 테스트 영상 확보
RQ3	라즈베리파이 안에서 추론하면 1초 간격 판정이 되는가	YOLO26s v2 벤치마크, 659장, 시간·메모리·온도 측정	불가. 1장 2,831.8ms, 최고 81.3°C

2. 실험 환경

표 2. 실험 환경

항목	내용
장치	Raspberry Pi 4 Model B Rev 1.2, 추론은 CPU 3스레드
카메라	Logitech C270 (USB)
송출	GStreamer RTSP 서버 (rtsp://<라즈베리파이 IP>:8554/camera)
모델	YOLO26s v2 (파라미터 10.0M), 입력 960, JPEG 입력
평가 영상	1차 룸 영상 960×540, 12fps, 약 22분
표본	2초 간격 659장(속도 주 지표), 보조 평가 포함 767장. 워밍업 1회 제외
측정일	2026-10-21

3. 카메라 송출과 PC 수신 추론 (1주차)

먼저 라즈베리파이에 연결한 USB 웹캠 영상을 네트워크로 내보내는 송출 모듈을 HTTP/MJPEG 방식으로 만들었다. PC에서는 이 영상을 받아 YOLO로 사람을 찾는 테스트 스크립트를 작성해, 카메라 → 네트워크 → PC 판정으로 이어지는 흐름을 확인했다. 또 테스트 영상을 확보하려고 수신 스트림을 파일로 저장하는 녹화 기능을 넣었다. 라즈베리파이와 C270 웹캠으로 찍은 영상(18초, 3명 공부 중)은 이후 사전학습 YOLO 16종 비교의 시험 영상으로 쓰였다.

4. H.264 RTSP 송출로 전환 (2주차)

2주차에는 송출 방식을 HTTP/MJPEG에서 H.264로 압축한 RTSP 송출로 바꿨다(pi_camera_rtsp.py). MJPEG는 화면을 한 장씩 JPEG로 보내 데이터가 크지만, H.264는 앞뒤 화면의 차이를 압축해 같은 화질을 훨씬 적은 데이터로 보낸다. 또 운영에 쓸 카메라(Tapo C210)도 RTSP로 영상을 주기 때문에, 서버는 시험용 라즈베리파이와 실제 카메라를 같은 방식으로 받을 수 있다.

표 3. 송출 파이프라인 (pi_camera_rtsp.py 기본값)

단계	내용
카메라 입력	C270에서 MJPEG으로 받는다. USB 전송량을 줄이려고 압축된 형식으로 받은 뒤 한 번 디코딩한다
인코딩	소프트웨어 x264(ultrafast, zerolatency, 키프레임 2초마다) 또는 하드웨어 v4l2h264enc 중 선택, 2,000kbps
송출	GStreamer RTSP 서버, 포트 8554, 주소 /camera
화면	1280×720, 15fps (실행 옵션으로 변경 가능)
보안	접근 제어와 암호화가 없어, 신뢰할 수 있는 교내 사설망에서만 쓴다

이 스트림은 팀의 실시간 인식 화면(realtime/live.py)이 TCP로 받아, 1280×720·15fps 영상에 YOLO26s를 돌려 초당 17.8장을 표시했다.

5. 라즈베리파이 온디바이스 추론 벤치마크 (2주차)

1차 회의에서는 룸마다 라즈베리파이를 두고 그 안에서 직접 추론하는 안을 검토했다. 이 안이 가능한지 숫자로 판단하려고, 최종 모델 YOLO26s v2를 라즈베리파이 4B에서 직접 돌리는 벤치마크 프로그램(pi_team_benchmark v3)을 만들었다. 팀과 같은 모델·입력 크기·판정 규칙으로 같은 영상을 돌려 연산 장치만 다르게 했고, 모델 로딩·표본 준비·워밍업·결과 저장은 처리 시간에서 뺐다.

표 4. 처리 속도 (659장)

측정 범위	평균	P95	최대
모델 내부 추론	2,800.0ms	2,832.4ms	2,862.8ms
predict 전체 (주 지표)	2,831.8ms	2,863.8ms	2,899.8ms
입력 읽기·predict·박스 추출	2,832.4ms	2,864.2ms	2,900.6ms

환산 처리량은 초당 0.353장, 최대 메모리는 620.5MiB, 최고 온도는 81.3°C였다. 그림 1처럼 처리 시간은 31분 내내 2.8초 안팎으로 일정했지만, CPU 온도는 62.3°C에서 시작해 약 10분 만에 80°C 근처까지 올라 그대로 머물렀다.

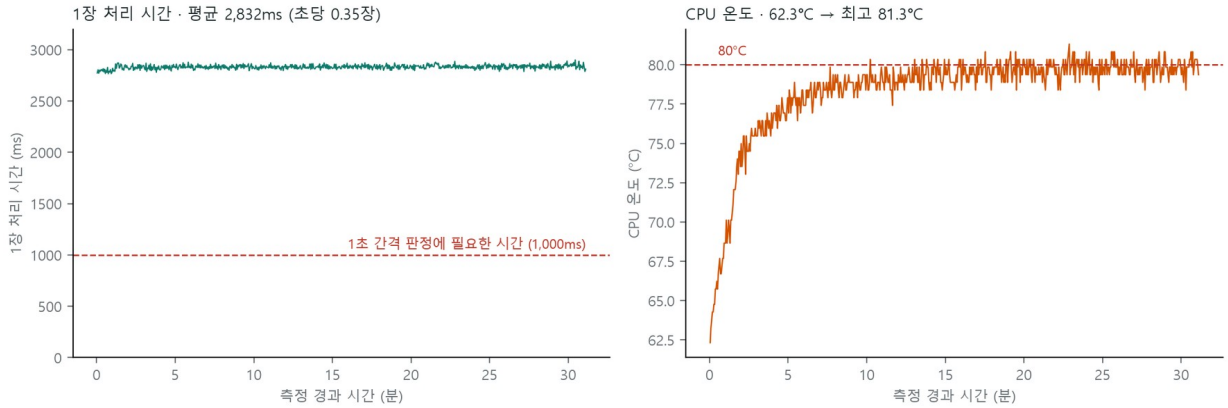


그림 1. 라즈베리파이 4B에서 YOLO26s v2로 659장을 연속 처리한 기록. 왼쪽은 1장 처리 시간(빨간 점선은 1초 간격 판정에 필요한 1,000ms), 오른쪽은 CPU 온도

표 5. 검출·판정 결과 (참조 기준)

항목	결과	채점 방식
테이블 (학습 외 뒤 20%)	100% (131/131)	최고 신뢰도 박스와 참조 박스의 IoU 0.5 이상
엎드린 사람	100% (84/84)	지정 영역의 사람 신뢰도 0.35 이상
앉은 사람	100% (108/108)	참조 영역 3곳의 일대일 검출 수
재실 판정	빨강 189 · 노랑 470 · 초록 0, 놓침 0 · 오탐 0	연속 2회 확인 · 30분 기준

원본 평가 코드와 정답 라벨이 없어, 영상을 직접 확인해 만든 참조 박스·영역으로 채점한 점수다. 같은 모델을 쓰므로, 라즈베리파이에서도 정확도가 떨어지지 않는다는 것을 확인하는 용도로 본다.

6. 결론

정확도는 서버와 같았지만, 세 가지 이유로 룸별 라즈베리파이 추론은 쓸 수 없다는 결론을 냈다.

- **속도:** 1장에 2,831.8ms가 걸려 1초 간격 판정(1,000ms)을 맞출 수 없다. 659장 모두 당시 표본 간격 2초보다도 길었고, GPU 서버(18.1ms)보다 약 156배 느리다.
- **발열:** 연속 처리 10분 만에 80°C 근처에 이르러, 하루 종일 돌리면 속도가 떨어지거나 고장 날 위험이 있다.
- **운영:** 룸마다 1대씩 14대를 따로 관리해야 하고, 초기 비용도 약 240만 원으로 교내 PC 1대보다 크다.

이 결과로 팀은 라즈베리파이를 판정 장치에서 제외하고, GPU가 있는 교내 PC 1대에서 14개 룸을 모두 판정하기로 했다. 라즈베리파이로 만든 RTSP 송출은 서버가 카메라 영상을 받는 방식을 미리 시험하는 데 쓰였다.

박서준 개인 연구 보고: 서버 판정 시스템 설계와 YOLO 검증

담당: 서버 판정 프로그램, YOLO 모델 비교·검증, 생성형 데이터 실험. 코드와 프레임별 결과는 GitHub 10_Project 폴더(https://github.com/gosu1/ERICA_X_MODULABS_CV/tree/main/10_Project)에 있고, 이 보고서의 모든 수치는 그 기록에서 가져왔다.

요약

사전학습 YOLO 15종(v8·11·26 계열의 n~x)을 우리 데이터 세 종류로 비교해, 파라미터 대비 성능이 가장 좋은 YOLO26s를 골랐다. YOLO26s(9.5M)는 파라미터가 6배인 YOLO11x(56.9M)와 핵심 지표가 같았고, 연산량은 9분의 1이었다. 판정 프로그램(server.py)은 2프레임 연속 확인과 끊긴 시간 제외 규칙을 넣어 만들었다. 정답을 표기한 1차 영상 659장에서 사전학습 YOLO26s는 656장(99.5%)이 맞았고, 팀의 파인튜닝 모델 v2는 학습에 쓰지 않은 구간 132장 중 131장이 맞았다. v2가 틀린 1장은 규칙이 의도한 2초 지연이었다. Stable Diffusion 인페인팅으로 만든 합성 사진 80장은 학습 데이터로는 효용이 낮았지만, 후드를 쓰고 엎드린 사람이라는 약점을 찾아 시범 수집의 우선순위를 정하는 근거가 됐다.

1. 연구 질문

서버 판정을 맡으면서 아래 여섯 가지 질문을 세우고, 각각 우리 데이터로 답했다.

표 1. 연구 질문, 방법, 결론

번호	연구 질문	방법	결론
RQ1	파라미터 대비 성능이 가장 좋은 사전학습 YOLO는 무엇인가	15종 × 우리 데이터 3종, COCO 공식 지표	YOLO26s
RQ2	사전학습 모델로 테이블을 잡아 감지 영역을 만들 수 있는가	15종의 COCO dining table 신뢰도	불가. 다각형 영역으로 대체
RQ3	판정 규칙이 실제 영상에서 의도대로 동작하는가	정답 659장과 프레임 단위 비교, 끊김·강제 종료 시험	26s 99.5%, v2 학습 외 구간 131/132
RQ4	생성형 데이터로 엎드린 사람 데이터를 채울 수 있는가	SD 인페인팅 80장, CLIP 필터, 15종 평가	학습 효용 낮음, 약점 탐색에 유효
RQ5	우리 환경과 비슷한 공개 데이터가 있는가	10종 조사(공식 페이지, 논문, 라이선스)	스터디룸 각도의 엎드림 데이터 없음
RQ6	카메라 영상을 지연 없이 실시간 판정할 수 있는가	최신 프레임만 유지하는 수신 구조	초당 17.8장 표시

2. 실험 환경과 데이터

표 2. 실험 환경

항목	내용
장치	M2 맥북 16GB, MPS(애플 GPU)
소프트웨어	Python 3.12, Ultralytics, OpenCV, diffusers 0.41(합성 실험)
입력 크기	960 (모든 실험 공통)
평가 데이터	① 웹캠 녹화 36장(17초, 3명 계속 착석) ② 1차 룸 영상 659장(22분, 2초 간격, 정답 표기) ③ 합성 엎드린 장면 40장(4절의 80장 중 사람이 엎드린 장면)
정답	썸네일을 보고 2초 단위로 표기. 0:00~3:30 사람 있음(191장), 3:32~19:06 짐만 남음(468장), 19:08 이후 사람 있음
측정일	2026-10-18~21

15종을 한 프로세스에 모두 올리면 메모리 16GB를 넘을 수 있어서, 모델마다 별도 프로세스로 실행하고 끝나면 메모리를 돌려받게 했다. 이때 MPS가 GPU 쪽에 잡는 메모리는 프로세스 최대 메모리 값에 잡히지 않는다는 점을 확인했다. 예를 들어 YOLO11x는 실제로 약 1.8GB를 쓴다.

3. 모델 비교: 파라미터 대비 성능 (RQ1)

판정은 가장 약한 프레임에서 깨진다. 그래서 평균 대신 최저 신뢰도를 주 지표로 쓰고, 놓친 장수를 함께 봤다. 파라미터, 연산량, COCO mAP는 Ultralytics 공식 표(COCO val2017, 입력 640)이고, 나머지는 우리 데이터에서 잰 값이다.

표 3. 사전학습 15종 비교 (기준 0.35, 1차 영상은 JPEG 프레임 기준)

모델	파라미터 (M)	GFLOPs	COCO mAP50-95	녹화 36장 사람 최저	1차 놓침 / 오탐	1차 사람 최저	합성 엎드림 검출	합성 최저
YOLOv8n	3.2	8.7	37.3	0.48	0 / 0	0.44	34/40	0.00
YOLOv8s	11.2	28.6	44.9	0.31	1 / 0	0.34	39/40	0.09
YOLOv8m	25.9	78.9	50.2	0.61	0 / 0	0.52	39/40	0.00
YOLOv8l	43.7	165.1	52.9	0.49	0 / 0	0.45	38/40	0.00
YOLOv8x	68.2	257.8	53.9	0.56	0 / 0	0.53	39/40	0.06
YOLO11n	2.6	6.5	39.5	0.33	1 / 0	0.00	39/40	0.34
YOLO11s	9.4	21.6	47.0	0.45	0 / 0	0.40	37/40	0.00
YOLO11m	20.1	68.1	51.5	0.58	0 / 0	0.61	39/40	0.08
YOLO11l	25.3	87.2	53.4	0.83	0 / 0	0.40	37/40	0.12
YOLO11x	56.9	195.3	54.7	0.71	0 / 0	0.47	40/40	0.38
YOLO26n	2.4	5.5	40.1	0.54	3 / 0	0.00	33/40	0.10
YOLO26s ★	9.5	20.9	47.8	0.81	0 / 0	0.47	40/40	0.36
YOLO26m	20.4	68.4	52.5	0.76	0 / 0	0.64	38/40	0.09
YOLO26l	24.8	86.8	54.4	0.88	0 / 0	0.54	39/40	0.14
YOLO26x	55.7	194.4	56.9	0.89	0 / 0	0.70	38/40	0.15

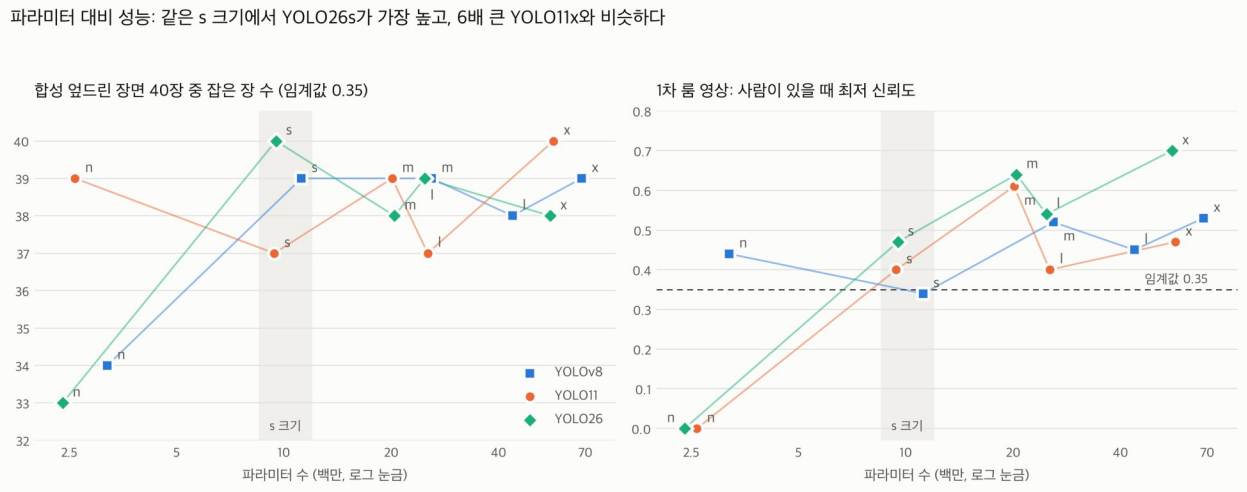


그림 1. 파라미터 수(로그 눈금)와 성능. 왼쪽은 합성 엎드린 장면 40장 중 잡은 장 수, 오른쪽은 1차 영상에서 사람이 있을 때의 최저 신뢰도

결과를 파레토 관점으로 보면, 다섯 지표(합성 검출 수, 합성 최저, 1차 사람 최저, 녹화 최저, COCO mAP) 모두에서 YOLO26s보다 파라미터가 적으면서 성능이 높은 모델은 없었다. 이 조건을 만족한 모델은 26s와, 가장 작아서 자동으로 만족하는 26n뿐이다.

- **26s 대 11x:** 파라미터를 6배 늘려도 합성 검출 수(40 대 40)와 1차 사람 최저(0.47 대 0.47)는 같았고, 녹화 36장 최저는 26s가 높았다(0.81 대 0.71). 연산량은 20.9 대 195.3 GFLOPs다.
- **n 크기:** 1차 영상에서 사람이 있는 프레임을 놓쳤다(11n 1장, 26n 3장).
- **같은 s 크기:** YOLO11s는 합성 엎드린 장면 3장을 놓쳤고 모두 후드나 패딩 차림이었다(0.00, 0.05, 0.25). YOLOv8s는 1차 영상에서 1장을 놓쳤다.

기준값은 오류 비용이 비대칭이라는 점을 고려해 0.30으로 정했다. 원본 프레임 기준으로 빈 방의 최고 점수는 0.12이고 합성 엎드린 장면의 최저 점수는 0.36이어서, 0.30은 두 값 사이에 있다. 26s가 놓친 3:30 프레임은 사람이 나가는 순간이라 노랑이 2초 일찍 켜질 뿐이다. 반면 엎드린 사람을 계속 놓치면 공부 중인 룸에 초록불이 켜지므로, 이쪽 여유를 더 크게 뒀다.

같은 프레임이라도 JPEG로 저장하면 점수가 달라지는지 확인했다. 659장 평균 차이는 작았지만, 기준값 근처의 경계 프레임에서는 판정이 갈렸다(표 4). 그래서 기준값 근처의 판단은 영상 원본 프레임으로 한다.

표 4. JPEG 압축의 영향 (3:30, 사람이 화면 끝으로 나가는 프레임)

모델	원본 프레임	JPEG 프레임	659장 평균 차이
YOLO26s	0.29	0.56	0.005
YOLO11s	0.41	0.48	0.003
YOLO11x	0.48	0.47	기록 없음

4. 테이블 검출과 감지 영역 (RQ2)

처음 계획은 테이블을 검출하고 그 박스를 넓혀 감지 영역으로 쓰는 것이었다. 그러나 COCO의 dining table은 대부분 옆에서 본 식탁이라, 높은 곳에서 대각선으로 내려다본 우리 테이블은 학습 분포 밖에 있었다. 녹화 36장에서 테이블 평균 신뢰도는 0.00~0.47에 머물렀고, 기준 0.35를 넘긴

것은 YOLO11l의 최고 박스(0.56)뿐이었다. 큰 모델일수록 오히려 낮아지는 경향도 보였다(11x 0.02, 26x 0.04, v8x 0.08).

표 5. 녹화 36장 결과: 속도, 사람, 테이블

모델	MPS ms/장	CPU ms/장	사람 최저	사람 놓침 (0.35)	테이블 평균 신뢰도
YOLOv8n	33	64	0.48	0/108	0.18
YOLOv8s	48	134	0.31	6/108	0.00
YOLOv8m	103	272	0.61	0/108	0.15
YOLOv8l	184	481	0.49	0/108	0.09
YOLOv8x	231	852	0.56	0/108	0.08
YOLO11n	24	76	0.33	4/108	0.15
YOLO11s	46	154	0.45	0/108	0.10
YOLO11m	107	354	0.58	0/108	0.28
YOLO11l	125	676	0.83	0/108	0.47
YOLO11x	241	911	0.71	0/108	0.02
YOLO26n	22	94	0.54	0/108	0.19
YOLO26s ★	53	204	0.81	0/108	0.15
YOLO26m	98	363	0.76	0/108	0.14
YOLO26l	117	449	0.88	0/108	0.14
YOLO26x	220	776	0.89	0/108	0.04

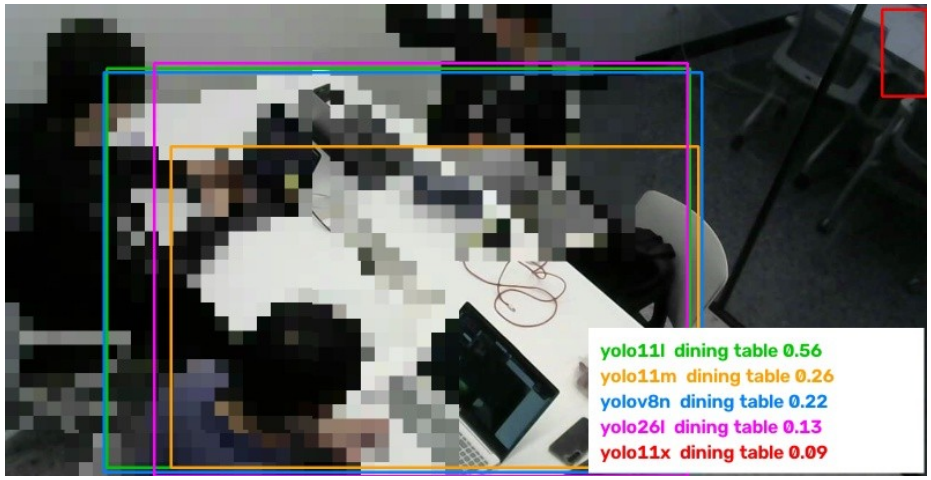


그림 2. 모델별로 가장 확실한 테이블 박스. YOLO11l(0.56)만 기준을 넘었고, YOLO11x(빨강)는 유리벽 너머 의자를 테이블로 잡았다

또 카메라가 테이블을 대각선으로 보기 때문에, 축에 맞춘 박스를 30% 넓히면 화면 거의 전체가 감지 영역이 된다. 그래서 감지 영역을 다각형으로 직접 지정하는 모듈(zone.py)을 만들었다. 사람 박스의 중심이 다각형 안에 있을 때만 재석으로 세므로, 유리벽 너머 복도를 지나가는 사람은 빠진다. 이후 팀이 파인튜닝한 v2가 테이블을 잡게 되면서 감지 영역은 SAM 테두리 방식으로 바뀌었고, 다각형 영역은 그 전까지의 판정과 검증 기준으로 쓰였다.



그림 3. 1차 영상의 다각형 감지 영역(초록). 왼쪽은 사람이 있는 2:00, 오른쪽은 빈 방인 10:00. 사람은 모자이크했다

5. 판정 알고리즘 설계 (server.py)

판정 프로그램은 프레임 수집, 사람 검출, 감지 영역 판정, 상태 판정의 네 단계로 이뤄진다(그림 4). 같은 코드가 녹화 영상과 실시간 스트림을 모두 처리하도록, 영상 파일은 영상 속 시각으로, 스트림은 실제 시각으로 시간을 잰다.

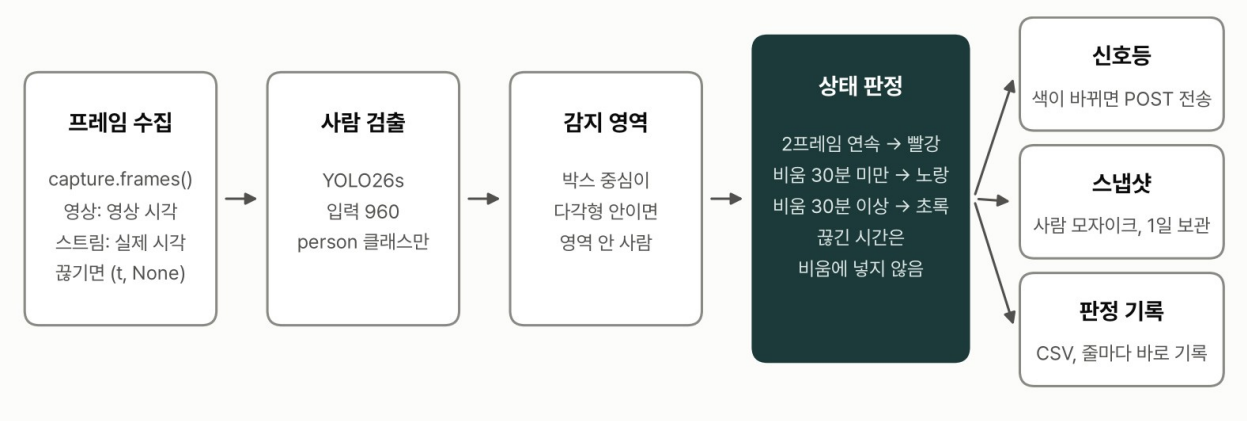


그림 4. server.py의 판정 흐름

표 6. 상태 정의

상태	조건	신호등
사용중	감지 영역 안 사람 신뢰도가 기준 이상인 프레임이 2번 연속	빨강
비움 30분 미만	마지막으로 사람을 확인한 뒤 비운 시간이 30분 미만	노랑
비움 30분 이상	비운 시간이 30분 이상(끊긴 시간 제외)	초록

규칙은 오류마다 피해가 다르다는 점에서 출발했다. 가장 피해가 큰 오류는 주인이 있는데 초록이 켜져 다른 학생이 짐을 치우는 경우다. 표 7의 결정은 모두 이 오류와 그 원인을 막기 위한 것이다.

표 7. 설계 결정과 근거

결정	막으려는 오류	근거
2프레임 연속 확인	순간 오검출 하나가 30분 타이머를 초기화해 알박기를 계속 가림	타이머는 사람이 한 번만 잡혀도 처음부터 다시 셈
끊긴 시간을 비움에서 제외	스트림 끊김을 사람 없음으로 세어 공부 중인 룸에 초록	끊김은 정보가 없는 상태이지 부재가 아님
박스 중심으로 영역 판정	유리벽 너머 복도 사람을 재석으로 셈	영역에 일부만 걸친 박스 제외
상태가 바뀐 순간만 스냅샷, 사람 모자이크, 1일 보관	영상 상시 저장에 따른 개인정보 위험	다툼이 생겼을 때 판정 근거만 확인
판정 기록을 한 줄씩 바로 저장	강제 종료 때 기록 유실	버퍼에 남은 기록이 사라지는 문제를 시험에서 발견
신호등은 색이 바뀔 때만 전송	불필요한 통신	HTTP POST {"color": "red" | "yellow" | "green"}

6. 판정 검증 (RQ3)

정답을 표기한 1차 영상으로 판정 결과를 프레임 단위로 비교했다. 이 영상의 앞 80%는 이후 v2 학습에 쓰였기 때문에, v2는 학습에 쓰지 않은 뒤쪽 20%(17:34 이후 132장)로만 평가했다. 사전학습 모델은 이 영상으로 학습하지 않았으므로 659장 전체로 평가했다.

표 8. server.py 검증 시험

시험	설정	결과
타이머 동작	1프레임 확인, 초록 기준 10분	빨강 191, 노랑 299, 초록 169. 정답과 불일치 0장. 마지막 검출(3:30) 10분 뒤인 13:30에 초록
YOLO11x	기준 0.35, 2프레임 확인, 30분	빨강 189, 노랑 470. 불일치 2장(0:00, 19:08 입장 첫 프레임)
사전학습 YOLO26s	기준 0.30, 2프레임 확인, 30분, 659장	656장 일치(99.5%). 불일치 3장(입장 첫 프레임 2장, 3:30 신뢰도 0.29)
파인튜닝 v2	기준 0.35, 학습 외 구간 132장	131장 일치. 불일치 1장(19:08 입장 첫 프레임). 사람 최저 0.79, 빈 룸 최고 0.00
스트림 끊김	임시 스트림 서버, 초록 기준 0.5분, 10초에 끊고 31초 뒤 재연결	끊긴 시간을 빼고 1:03에 초록. 신호등 수신기가 yellow, green 순서로 받음
모자이크	상태가 바뀐 순간의 스냅샷	화면 속 3명 모두 모자이크
강제 종료	실행 중 프로세스 종료	그때까지의 판정 기록이 CSV에 남음

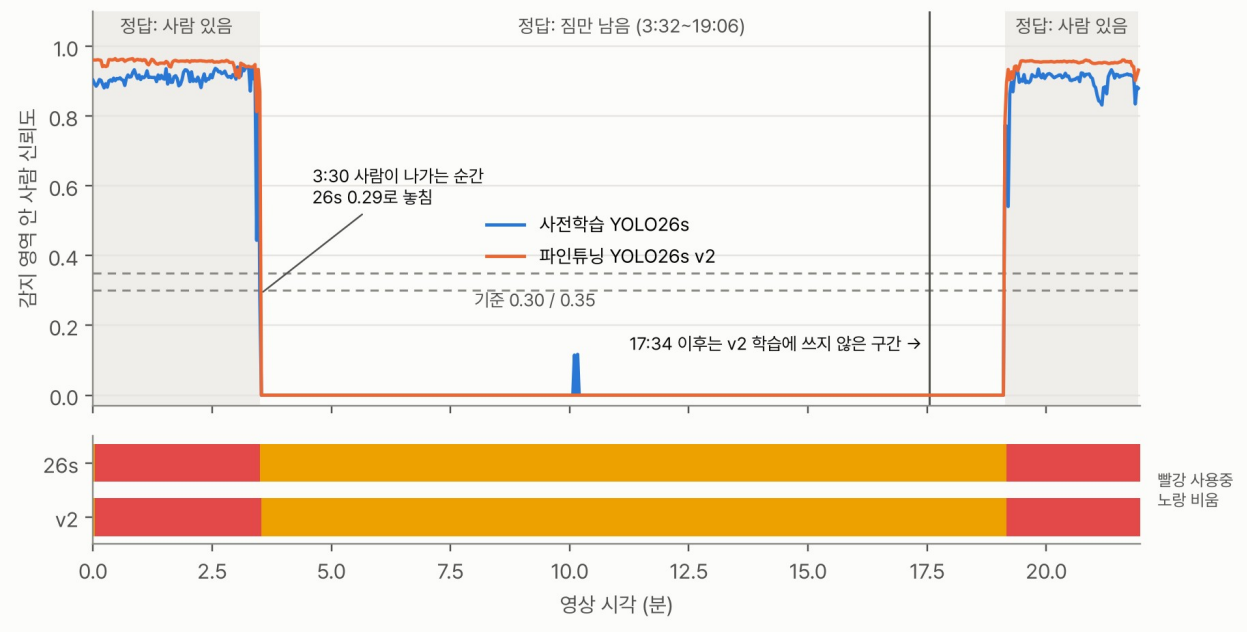


그림 5. 1차 영상의 감지 영역 안 사람 신뢰도(위)와 판정된 신호등 색(아래). 회색은 정답상 사람이 있는 구간

불일치는 두 종류로 나뉜다. 첫째는 입장 순간의 2초 지연이다. 2프레임 연속 확인 규칙이 의도한 동작이며, 순간 오검출을 막는 대가로 받아들였다. 둘째는 3:30의 검출 실패로, 사람이 화면 끝으로 나가며 몸이 잘린 프레임이다. 노랑이 2초 일찍 켜질 뿐 초록과는 무관하다. 결국 잘못된 초록은 어떤 시험에서도 나오지 않았다.

기준선과의 거리도 비교했다. 사람이 있을 때의 최저 신뢰도는 사전학습 26s가 0.29로 기준 0.30 아래까지 내려갔지만, v2는 학습 외 구간에서도 0.79로 기준 0.35보다 0.44 높았다. 빈 룸의 최고 점수도 26s는 0.12(10분 무렵 한 번)였고 v2는 0.00이었다. 파인튜닝이 사람 쪽과 빈 룸 쪽 양방향 모두에서 여유를 넓혔다는 뜻이다.

7. 추론 위치 비교 실험 설계

라즈베리파이 안에서 추론하는 방식과 서버에서 추론하는 방식을 공정하게 비교하도록 실험을 설계했다. 바꾸는 변인은 연산 장치 하나이고, 모델, 입력 크기, 기준값, 감지 영역, 판정 규칙, 판정 코드는 고정했다. 같은 영상을 양쪽에서 같은 명령으로 돌리면 장치 차이만 남는다.

표 9. 비교 지표와 제안 기준

지표	재는 법	제안 기준
판정 일치율	정답 구간과 신호등 색이 같은 프레임 비율	99% 이상
1장 처리 시간	판정 기록의 프레임별 시간(평균, 최대)	30초 이하
빨강 반응 시간	사람이 들어온 시각부터 첫 빨강까지	1분 이하
최대 메모리	실행 중 메모리 최댓값	기기 RAM의 70% 이하
끊김 시 동작	실행 중 Wi-Fi를 1분 끊고 신호등 기록	초록이 잘못 켜지지 않음

제안 기준의 처리 시간 30초는 빨강 반응 1분에 맞춘 값이다. 이후 팀은 판정 간격을 1초로 정했고, 라즈베리파이 실측(1장 2,832ms)이 이를 충족하지 못해 서버 방식을 택했다.

8. 생성형 데이터 실험 (RQ4)

엎드린 사람 데이터가 부족해서, 생성형 데이터로 채울 수 있는지 시험했다. Stable Diffusion v1.5 인페인팅으로 1차 영상의 빈 방 프레임(8:00)에 엎드린 사람을 그려 넣었다. 자리 4곳 중 A, B, C는 실제로 사람이 앉았던 자리이고, D는 아무도 앉지 않아 직접 정했다. 자리 주변 512×512만 잘라 그린 뒤 마스크 안쪽만 원본에 다시 붙였기 때문에, 그린 자리 밖은 실제 화면 그대로다.

사람이 그려졌는지는 CLIP(ViT-B/32)으로 먼저 걸렀다. YOLO와 다른 모델로 거르기 때문에, YOLO가 놓치는 이미지도 버려지지 않는다. 시험 생성 16장에서 기준 0.9가 사람의 눈 판정과 16장 모두 일치했다. 최종 판정은 사람이 눈으로 했고, 1장을 만드는 데 약 70초가 걸렸다.

표 10. 생성 회차별 결과 (엎드림 검출은 기준 0.35)

회차	바꾼 것	생성	사람 있음	엎드림	엎드림 검출 **11x · 26l · 26s**
시험	공통 지시문, 옷차림 4가지	16	5	3	3 · 3 · 3
2차	자리별 방향, 부정 지시문, 남녀 × 옷차림	32	24	16	16 · 16 · 16
어려운 장면	외투·후드·담요로 머리와 몸을 가림, D 마스크를 책상 쪽으로 이동	32	30	21	21 · 20 · 21
합계		80	59	40	40 · 39 · 40

- **지시문 효과:** 자리별 방향을 지시문에 넣자, 뒷모습이 보이는 C 자리에 사람이 그려진 비율이 0/4에서 8/8로 올랐다.
- **마스크 위치:** D 자리는 2차에서도 1/8이었지만, 마스크를 책상 가장자리 쪽으로 옮기자 7/8이 됐다. 사람이 놓일 책상 면이 마스크 안에 있어야 사람을 그린다.
- **CLIP 거르기:** 2차에서 엎드린 16장을 모두 남기고 사람 없는 8장을 모두 버렸다. 깨어 있는 자세 4장과 깨진 이미지 3장은 거르지 못해, 사람의 검수가 필요했다.
- **약점 발견:** 후드를 쓰고 흰 베개에 엎드린 장면(A_5)을 YOLO26l은 0.14로 놓쳤고, YOLO11x는 0.38로 겨우 잡았다. YOLO26s는 0.42로 잡았다.
- **오탐 확인:** 의자에 외투만 걸린 장면(A_2)은 어떤 모델도 사람으로 잡지 않았다.



그림 6. 어려운 장면 32장. 사람은 모두 Stable Diffusion이 그린 가상 인물이다. 노란 숫자는 CLIP 점수와 YOLO11x, 26l 신뢰도

결론적으로 합성 데이터는 학습에 쓰지 않았다. 첫째, 80장 중 YOLO가 놓친 엎드린 장면은 1장뿐이라, 학습에 쓸 어려운 표본 1장을 얻는 데 약 93분(80장 × 70초)이 들었다. 둘째, 실제 약점인 천장 각도의 테이블은 합성 사진으로 풀 수 없었다. 대신 합성 사진은 약점을 찾는 도구로 쓸모가 있었고, 그 결과 후드를 쓰고 엎드린 장면을 시범 수집에서 먼저 모으기로 했다. 합성 사진의 검출률은 실제 재현율이 아니므로, 모델 선택과 기준값은 실제 영상으로 정했다.

9. 공개 데이터셋 조사 (RQ5)

우리 영상과 비슷한 공개 데이터가 있으면 평가를 보강할 수 있어서, 공식 페이지, 논문, 저장소, 라이선스 문서를 직접 확인해 10종을 조사했다. 데이터 파일은 내려받지 않았다.

표 11. 공개 데이터셋 비교 (기준: 방 모서리 높은 곳에서 테이블 1개와 의자 4개를 내려다보는 카메라)

데이터셋	장면과 카메라 각도	규모	라이선스	적합도
Edinburgh Office Monitoring	대학 사무실 4곳, 방 위쪽 고정 카메라	20일, 456,714장	CC BY-NC-SA 3.0	높음
Employee Sleeping (Roboflow)	사무실 책상에 엎드린 사람, CCTV 각도	3,042장	CC BY 4.0 표기, 출처 불명	중간
SCB-Dataset (3-U)	교실, 정면·측면·후면	13,330장, 3-U 671장	비상업 학술용	중간
CAVIAR	연구소 로비, 가방 두고 가기, 거의 바로 위	28클립	CC BY-SA	중간
PIROPO	실내 방 2곳, 벽·천장 카메라	10만 장 이상	연구 목적 무료	중간
Toyota Smarthome	아파트 일상, 방 모서리 위쪽	16,115클립	비상업, 재배포 금지	중간
CEPDOF	작은 강의실, 천장 어안	25,504장	비상업 또는 ODbL	낮음
EduNet	인도 교실, 여러 각도	7,851클립	표기 없음	낮음
Classroom Behavior	교실 학생, 각도 섞임	4,881장	CC BY 4.0 표기, 출처 불명	낮음
ABODA	물건이 버려진 장면	영상 11개	표기 없음	낮음

가장 가까운 것은 Edinburgh Office Monitoring이다. 사무실 위쪽 고정 카메라로 20일을 1초 1장 간격으로 찍었고, 빈 방과 앉은 사람에 프레임별 라벨이 있어 빈 방 오검출을 함께 볼 수 있다. 엎드린 사람은 Employee Sleeping과 SCB-Dataset의 leaning over the table 라벨이 가깝지만, 전자는 이미지 출처 설명이 없고 후자는 허락 없는 배포를 금지한다. 스터디룸 같은 작은 방에서 책상에 엎드린 사람을 방 모서리 각도로 찍은 공개 데이터는 찾지 못했다. 그래서 평가와 학습에는 우리 룸에서 직접 모은 데이터가 필요하다고 판단했다.

10. 실시간 수신 구조 (RQ6)

라즈베리파이가 RTSP로 보내는 카메라 영상을 노트북에서 받아 실시간으로 사람을 표시하는 프로그램(realtime/live.py)을 만들었다. 수신과 추론을 분리한 것이 핵심이다. 수신 스레드는 프레임을 계속 받아 가장 최근 1장만 들고 있고, 추론 쪽은 그 1장만 가져간다. 그래서 추론이 수신보다 느려도 지연이 쌓이지 않는다. RTSP는 TCP로 받는다. UDP로 받으면 Wi-Fi에서 패킷이 빠져 깨진 프레임이 들어오기 때문이다. 수신이 실패하면 2초 뒤 다시 연결한다. 이 구조로 1280×720·15fps 영상에 YOLO26s를 돌려 초당 17.8장을 표시했다.

11. 재현성과 개인정보 보호

다른 팀원이 같은 결과를 다시 낼 수 있도록 코드, 프레임별 결과, 실험 기록을 GitHub에 올렸다.

표 12. GitHub 10_Project 구성

파일	역할
server.py	판정 본체(상태 판정, 신호등 전송, 스냅샷, 판정 기록)
capture.py	영상·스트림 프레임 수집, 끊김 감지와 재연결
zone.py	다각형 감지 영역 지정과 영역 판정 함수
compare.py	15종을 모델마다 별도 프로세스로 돌리는 비교 실험
realtime/live.py	RTSP 실시간 인식 화면
synthetic/	합성 데이터 생성·거르기 코드, 자리 좌표, 15종 평가 결과
1차_영상_결과.csv	15종 × 659장 프레임별 결과(9,885행)
EXPERIMENTS.md	모든 실험의 설정, 수치, 해석
DATASETS.md	공개 데이터셋 10종 조사
image/	실험 이미지 9장과 설명
CLAUDE.md	인수인계 문서(결정과 이유, 실행 방법, 주의 사항)

저장소가 공개이므로 개인정보 보호를 처음부터 작업 규칙에 넣었다. 실제 사람이 나오는 이미지는 YOLO11x로 사람을 찾고 박스를 사방 25% 넓힌 뒤 24픽셀 단위로 모자이크했다. 팀원 얼굴이 나오는 영상, 스냅샷, 모델 가중치는 저장소에 올라가지 않도록 업로드 제외 목록(.gitignore)으로 막았다.

12. 그 밖의 기여

- **발표 자료:** 실험 결과를 반영한 PPT 수정본(18장)을 만들고, 검증 결과 슬라이드와 부록 2장을 추가했다.
- **예상 질의응답:** 결과물 전체를 평가자 관점에서 검토해 예상 질문 14개(Q17~Q30)와 답변을 정리했다.
- **학습·평가 데이터 분리 점검:** 판정 검증 영상의 앞 80%가 v2 학습에 쓰였다는 점을 확인하고, v2 성능을 학습 외 구간으로 다시 계산했다.

13. 결론

이번 작업의 기여는 네 가지로 정리된다.

- **모델 선택:** 파라미터 대비 성능 분석으로, YOLO11x와 핵심 지표가 같으면서 파라미터는 6분의 1, 연산량은 9분의 1인 YOLO26s를 골랐다.
- **오류 비용 반영:** 잘못된 초록이 가장 큰 피해라는 점을 판정 규칙(2프레임 연속 확인, 끊긴 시간 제외)과 평가 지표(최저 신뢰도)에 반영했다.
- **검증 방법:** 정답 영상과 프레임 단위로 비교하고, 학습에 쓴 구간을 분리해 평가했다. 어떤 시험에서도 잘못된 초록은 나오지 않았다.
- **생성형 데이터의 재정의:** 합성 사진을 학습 데이터가 아니라 약점을 찾는 도구로 써서, 현장 수집의 우선순위를 정했다.